



**NUST COLLEGE OF  
ELECTRICAL AND MECHANICAL ENGINEERING**



**IOT BASED PREDICTIVE MAINTENANCE SYSTEM FOR  
INDUSTRIAL MACHINERY**

**A PROJECT REPORT**

**DE-44 (DC & SE)**

***Submitted by***

NS Faiqa Ummarah Ahmad

NS Shafla Rehman

NS Syed Suleman Ali

ASC Ibrahim Umar

**BACHELORS  
IN  
COMPUTER ENGINEERING  
YEAR  
2026**

**PROJECT SUPERVISOR**

DR. ALI HASSAN

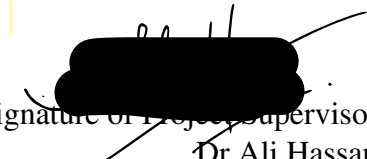
DR. FARHAN HUSSAIN

DR. SHOAB KHAN

DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING  
COLLEGE OF ELECTRICAL & MECHANICAL ENGINEERING  
NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY, ISLAMABAD, PAKISTAN

# Certification

This is to certify that Faiqa Ummarah Ahmad [405052], Shafla Rehman [412780], Ibrahim Umar [432295] and Syed Suleman Ali [410541] have successfully completed the final project IoT Based Predictive Maintenance System for Industrial Machinery, at the NUST College of Electrical and Mechanical Engineering, to fulfill the partial requirement of the Bachelor's degree in Computer Engineering.

  
Signature of Project Supervisor  
Dr Ali Hassan  
Professor

# Sustainable Development Goals (SDGs)

| SDG No | Description                     | SDG No | Description                              |
|--------|---------------------------------|--------|------------------------------------------|
| SDG 1  | No Poverty                      | SDG 9  | Industry, Innovation, and Infrastructure |
| SDG 2  | Zero Hunger                     | SDG 10 | Reduced Inequalities                     |
| SDG 3  | Good Health and Well Being      | SDG 11 | Sustainable Cities and Communities       |
| SDG 4  | Quality Education               | SDG 12 | Responsible Consumption and Production   |
| SDG 5  | Gender Equality                 | SDG 13 | Climate Change                           |
| SDG 6  | Clean Water and Sanitation      | SDG 14 | Life Below Water                         |
| SDG 7  | Affordable and Clean Energy     | SDG 15 | Life on Land                             |
| SDG 8  | Decent Work and Economic Growth | SDG 16 | Peace, Justice and Strong Institutions   |
|        |                                 | SDG 17 | Partnerships for the Goals               |



Sustainable Development Goals

# Complex Engineering Problem

## Range of Complex Problem Solving

|   | Attribute                                                               | Complex Problem                                                                                                                                                                                  |   |
|---|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 1 | Range of conflicting requirements                                       | Involve wide-ranging or conflicting technical, engineering and other issues.                                                                                                                     | ✓ |
| 2 | Depth of analysis required                                              | Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models.                                                                                    | ✓ |
| 3 | Depth of knowledge required                                             | Requires research-based knowledge much of which is at, or informed by, the forefront of the professional discipline and which allows a fundamentals-based, first principles analytical approach. | ✓ |
| 4 | Familiarity of issues                                                   | Involve infrequently encountered issues                                                                                                                                                          | ✓ |
| 5 | Extent of applicable codes                                              | Are outside problems encompassed by standards and codes of practice for professional engineering.                                                                                                | ✓ |
| 6 | Extent of stakeholder involvement and level of conflicting requirements | Involve diverse groups of stakeholders with widely varying needs.                                                                                                                                |   |
| 7 | Consequences                                                            | Have significant consequences in a range of contexts.                                                                                                                                            | ✓ |
| 8 | Interdependence                                                         | Are high level problems including many component parts or sub-problems                                                                                                                           | ✓ |

## Range of Complex Problem Activities

|   | Attribute                                   | Complex Activities                                                                                                                                |   |
|---|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 1 | Range of resources                          | Involve the use of diverse resources (and for this purpose, resources include people, money, equipment, materials, information and technologies). | ✓ |
| 2 | Level of interaction                        | Require resolution of significant problems arising from interactions between wide ranging and conflicting technical, engineering or other issues. | ✓ |
| 3 | Innovation                                  | Involve creative use of engineering principles and research-based knowledge in novel ways.                                                        | ✓ |
| 4 | Consequences to society and the environment | Have significant consequences in a range of contexts, characterized by difficulty of prediction and mitigation.                                   | ✓ |
| 5 | Familiarity                                 | Can extend beyond previous experiences by applying principles-based approaches.                                                                   | ✓ |

*Dedicated to our family, friends, and  
teachers for their unwavering support and  
guidance.*

# Acknowledgment

We are deeply grateful to our supervisors, Dr. Ali Hassan, Dr. Shoab Khan, and Dr. Farhan Hussain, for their constant guidance, motivation, and kindness. They helped us overcome every obstacle and believed in us even when we struggled. They have left a lasting mark on our lives.

We also thank our families and friends for their endless love, patience, and support. Without them standing beside us, this journey would have been much harder. We are forever grateful.

# Abstract

Industrial systems commonly rely on reactive or scheduled maintenance strategies, where faults are addressed only after failure or at fixed intervals. These approaches often fail to detect early-stage degradation, as subtle changes in vibration, temperature, or current go unnoticed during operation. As a result, minor defects can escalate into critical failures, causing unplanned downtime, production losses, and increased costs. Predictive maintenance can reduce unplanned downtime by up to 30–50%; however, existing solutions are often expensive and complex, limiting their adoption in small- and medium-scale industries.

To address these challenges, this project presents an IoT-based predictive maintenance system for continuous monitoring and early fault detection in rotating machinery. The system integrates edge sensing, cloud-based processing, and machine learning into a unified pipeline. Sensor data is collected in real time using an ESP32-based setup and transmitted to the cloud, where it is processed and analyzed through an inference service.

An anomaly detection approach based on an LSTM Autoencoder is used to identify deviations from normal behavior, combined with a consecutive detection strategy to improve reliability. When anomalies are detected, a classification model determines the fault type, and the results are visualized through a web-based dashboard.

The system demonstrates effective performance in identifying faults at an early stage, providing advance warning more than 10 days before actual failure in experimental scenarios. This enables proactive maintenance scheduling, reduces downtime, and improves operational efficiency. These results highlight the potential of the proposed solution as a practical, low-cost approach for enhancing reliability in industrial environments.

**Keywords:** IoT, Predictive Maintenance, Anomaly Detection, Early Fault Detection, LSTM Autoencoder, CNN, Industrial IoT, Downtime Reduction

# Contents

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>Acknowledgment</b>                                           | <b>v</b>   |
| <b>Abstract</b>                                                 | <b>vi</b>  |
| <b>Contents</b>                                                 | <b>ix</b>  |
| <b>List of Figures</b>                                          | <b>x</b>   |
| <b>List of Tables</b>                                           | <b>xii</b> |
| <b>Chapter 1: Introduction</b>                                  | <b>1</b>   |
| 1.1 Motivation . . . . .                                        | 2          |
| 1.2 Problem Statement . . . . .                                 | 3          |
| 1.3 Scope . . . . .                                             | 4          |
| 1.4 Aims and Objectives . . . . .                               | 5          |
| 1.5 Outcomes . . . . .                                          | 5          |
| 1.6 Report Organization . . . . .                               | 6          |
| <b>Chapter 2: Background &amp; Related Work</b>                 | <b>8</b>   |
| 2.1 Bearing Fault Diagnosis: Fundamentals and Types . . . . .   | 8          |
| 2.2 Related Work . . . . .                                      | 9          |
| 2.2.1 Physics-Based and Signal Processing Approaches . . . . .  | 10         |
| 2.2.2 Traditional Machine Learning . . . . .                    | 10         |
| 2.2.3 Deep Learning for Fault Classification . . . . .          | 11         |
| 2.2.4 Unsupervised Anomaly Detection . . . . .                  | 12         |
| 2.3 Gaps in Existing Research . . . . .                         | 12         |
| <b>Chapter 3: Material &amp; Component</b>                      | <b>14</b>  |
| 3.1 Device Hardware Components . . . . .                        | 14         |
| 3.1.1 Central Processing Unit . . . . .                         | 14         |
| 3.1.2 Sensors . . . . .                                         | 16         |
| 3.1.3 Sensor Connectivity . . . . .                             | 18         |
| 3.1.4 Device Enclosure . . . . .                                | 19         |
| 3.2 Software . . . . .                                          | 19         |
| 3.3 Data Collection . . . . .                                   | 20         |
| 3.3.1 Test Rig Description . . . . .                            | 20         |
| 3.3.2 NASA IMS Bearing Dataset . . . . .                        | 21         |
| 3.3.3 CWRU Dataset and Artificial Degradation Dataset . . . . . | 22         |
| 3.3.4 Dataset Validation Strategy . . . . .                     | 24         |
| <b>Chapter 4: Methodology</b>                                   | <b>26</b>  |
| 4.1 System Overview . . . . .                                   | 26         |
| 4.2 Edge Data Acquisition Layer . . . . .                       | 27         |
| 4.2.1 ESP32 Microcontroller . . . . .                           | 28         |

|                   |                                                     |           |
|-------------------|-----------------------------------------------------|-----------|
| 4.2.2             | Sensor Configuration & Signal Acquisition . . . . . | 28        |
| 4.2.3             | Secure MQTT Transmission . . . . .                  | 29        |
| 4.3               | Cloud Ingestion Pipeline . . . . .                  | 30        |
| 4.3.1             | Azure IoT Hub . . . . .                             | 30        |
| 4.3.2             | Event Hub-Triggered Azure Function . . . . .        | 30        |
| 4.3.3             | Azure Blob Storage . . . . .                        | 31        |
| 4.4               | Machine Learning Pipeline . . . . .                 | 31        |
| 4.4.1             | Data Preprocessing . . . . .                        | 32        |
| 4.4.2             | Feature Extraction and Windowing . . . . .          | 32        |
| 4.4.3             | Normalization and Sequence Creation . . . . .       | 32        |
| 4.4.4             | LSTM Autoencoder for Anomaly Detection . . . . .    | 32        |
| 4.4.5             | K-Consecutive Fault Detection Mechanism . . . . .   | 34        |
| 4.4.6             | CNN-Based Fault Classification . . . . .            | 34        |
| 4.4.7             | Validation Strategy . . . . .                       | 35        |
| 4.4.8             | Final Output Generation . . . . .                   | 37        |
| 4.5               | Serverless API Layer . . . . .                      | 38        |
| 4.6               | Frontend Architecture . . . . .                     | 38        |
| 4.6.1             | API Integration & Data Sources . . . . .            | 38        |
| 4.6.2             | Data Mapping & Transformation Layer . . . . .       | 39        |
| <b>Chapter 5:</b> | <b>Deployment and Validation</b>                    | <b>40</b> |
| 5.1               | Deployment Overview . . . . .                       | 40        |
| 5.2               | Containerization of the Inference API . . . . .     | 40        |
| 5.2.1             | Docker Image Configuration . . . . .                | 40        |
| 5.2.2             | Application Dependencies . . . . .                  | 41        |
| 5.2.3             | Model Artifact Loading at Startup . . . . .         | 41        |
| 5.2.4             | FastAPI Endpoints . . . . .                         | 42        |
| 5.3               | Azure Cloud Deployment . . . . .                    | 43        |
| 5.3.1             | Azure Container Registry . . . . .                  | 43        |
| 5.3.2             | Azure Container Apps . . . . .                      | 43        |
| 5.4               | Azure Functions Deployment . . . . .                | 44        |
| 5.5               | Frontend Deployment . . . . .                       | 44        |
| 5.6               | Dashboard Walkthrough . . . . .                     | 45        |
| 5.6.1             | Dashboard Header . . . . .                          | 45        |
| 5.6.2             | System Status Panel . . . . .                       | 45        |
| 5.6.3             | Machine Health Metrics Panel . . . . .              | 46        |
| 5.6.4             | Rolling Statistics Panel . . . . .                  | 47        |
| 5.6.5             | Live Sensor Telemetry Charts . . . . .              | 47        |
| 5.6.6             | Reconstruction Error Chart . . . . .                | 47        |
| 5.7               | Model Performance Evaluation . . . . .              | 48        |
| 5.7.1             | Evaluation on Developed Dataset . . . . .           | 48        |
| 5.7.2             | Evaluation on NASA IMS Dataset . . . . .            | 52        |
| 5.7.3             | Real-Time Hardware Validation . . . . .             | 54        |
| 5.7.4             | Discussion . . . . .                                | 55        |
| 5.8               | System Validation . . . . .                         | 56        |
| 5.8.1             | Inference API Validation . . . . .                  | 56        |
| 5.8.2             | End-to-End Pipeline Validation . . . . .            | 57        |

|                                              |           |
|----------------------------------------------|-----------|
| <b>Chapter 6: Conclusion and Future Work</b> | <b>59</b> |
| 6.1 Conclusion . . . . .                     | 59        |
| 6.2 Future Work . . . . .                    | 59        |
| <b>References</b>                            |           |

# List of Figures

|             |                                                                                                                                                                                                                   |    |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1    | Sustainable Development Goals . . . . .                                                                                                                                                                           | ii |
| Figure 2.1  | Schematic representation of bearing fault types: (a) Inner race fault, (b) Outer race fault, (c) Rolling element fault . . . . .                                                                                  | 9  |
| Figure 3.1  | ESP32 WROOM-32 Microcontroller . . . . .                                                                                                                                                                          | 16 |
| Figure 3.2  | ADXL345 Accelerometer (Vibration Sensor) . . . . .                                                                                                                                                                | 17 |
| Figure 3.3  | SCT-013 30A/1V non-invasive current sensor . . . . .                                                                                                                                                              | 17 |
| Figure 3.4  | MAX6675 Module and K-type Thermocouple . . . . .                                                                                                                                                                  | 18 |
| Figure 3.5  | Device in 3D-printed casing . . . . .                                                                                                                                                                             | 19 |
| Figure 3.6  | Device in 3D-printed casing . . . . .                                                                                                                                                                             | 19 |
| Figure 3.7  | Bearing Fault Conditions Used in Experiments . . . . .                                                                                                                                                            | 21 |
| Figure 3.8  | Test Rig Assembly . . . . .                                                                                                                                                                                       | 21 |
| Figure 3.9  | Simulator used for NASA run-to-failure experiment . . . . .                                                                                                                                                       | 22 |
| Figure 3.10 | Detailed structure of CWRU dataset . . . . .                                                                                                                                                                      | 23 |
| Figure 3.11 | Data Collection Conditions . . . . .                                                                                                                                                                              | 24 |
| Figure 3.12 | Sample Concatenated Data in order (normal vibration data-low severity outer race fault vibration data-high severity outer race fault vibration data) . . . . .                                                    | 24 |
| Figure 4.1  | System-Level Overview: From Edge Device Deployment to Dashboard Visualization . . . . .                                                                                                                           | 27 |
| Figure 4.2  | End-to-End Cloud Deployment Architecture of the IoT-Based Predictive Maintenance System . . . . .                                                                                                                 | 27 |
| Figure 5.1  | Model Artifacts Stored in Azure Blob Storage . . . . .                                                                                                                                                            | 42 |
| Figure 5.2  | Real-Time ML Inference Pipeline diagram . . . . .                                                                                                                                                                 | 43 |
| Figure 5.3  | Inference API Deployed on Azure Container Apps . . . . .                                                                                                                                                          | 43 |
| Figure 5.4  | Dashboard Header with Live Stream Indicator . . . . .                                                                                                                                                             | 45 |
| Figure 5.5  | System Status Panel Showing Inner Race Fault Detection . . . . .                                                                                                                                                  | 46 |
| Figure 5.6  | Machine Health Metrics Panel . . . . .                                                                                                                                                                            | 46 |
| Figure 5.7  | Rolling Statistics Panel Caption . . . . .                                                                                                                                                                        | 47 |
| Figure 5.8  | Machine Health Metrics Panel . . . . .                                                                                                                                                                            | 47 |
| Figure 5.9  | Reconstruction Error Chart with Fault Detection Markers . . . . .                                                                                                                                                 | 48 |
| Figure 5.10 | Reconstruction error highlighting inner race fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 50.64 hours prior to failure and CNN confirming the fault type. . . . . | 49 |

|                                                                                                                                                                                                                                     |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 5.11 Reconstruction error highlighting outer race fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 50.33 hours prior to failure and CNN confirming the fault type. . . . .       | 50 |
| Figure 5.12 Reconstruction error highlighting rolling element fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 100.72 hours prior to failure and CNN confirming the fault type. . . . . | 51 |
| Figure 5.13 Reconstruction error for Set 1 Bearing 3 (inner race defect). Fault confirmed approximately 16.01 days before failure. . . . .                                                                                          | 52 |
| Figure 5.14 Reconstruction error for Set 1 Bearing 4 (rolling element defect). Fault confirmed approximately 18.24 days before failure. . . . .                                                                                     | 53 |
| Figure 5.15 Reconstruction error for Set 2 Bearing 1 (outer race defect). Fault confirmed approximately 3.05 days before failure. . . . .                                                                                           | 53 |
| Figure 5.16 Reconstruction error for a healthy bearing, showing a gradual increase over time that remains below the fault threshold and does not trigger fault confirmation. . . . .                                                | 54 |
| Figure 5.17 Real-time reconstruction error under normal operating condition. The error remains consistently below the threshold throughout and no fault is confirmed. . . . .                                                       | 55 |
| Figure 5.18 Real-time reconstruction error under faulty operating condition. The error exceeds the threshold from the first window and a fault is confirmed immediately after 10 consecutive anomalous sequences. . . . .           | 55 |

# List of Tables

|           |                                           |    |
|-----------|-------------------------------------------|----|
| Table 5.1 | FastAPI Inference API Endpoints . . . . . | 42 |
| Table 5.2 | End-to-End Validation Results . . . . .   | 57 |

# Chapter 1

## Introduction

Industrial machinery forms the backbone of modern production systems, and keeping that machinery running reliably remains one of the most persistent challenges faced by engineers and plant managers across every manufacturing sector. When a critical machine fails unexpectedly, consequences extend well beyond the immediate repair: production lines halt, delivery commitments are missed, and secondary damage to surrounding equipment can compound the original problem.

For much of industrial history, maintenance was either reactive or scheduled on a calendar basis regardless of actual machine condition. Both approaches have proven inadequate. The shift toward data-driven, condition-based maintenance, made possible by advances in sensing, wireless communication, and machine learning, represents one of the most significant developments in modern industrial operations. Machines rarely fail without warning; physical deterioration leaves measurable traces in vibration, temperature, and current signals long before it progresses to catastrophic failure. If those traces can be detected early enough, maintenance can be scheduled proactively, eliminating unplanned downtime while also avoiding unnecessary servicing.

This project designs, implements, and validates an **IoT-based predictive maintenance system** for rotating industrial machinery, with a particular focus on rolling element bearings. The system integrates low-cost edge sensing hardware, a secure cloud data pipeline,

machine learning-based fault detection and classification, and a real-time monitoring dashboard into a single, coherent architecture built entirely from commercially available components and open-source software.

## 1.1 Motivation

The financial and operational cost of unplanned equipment failure is well documented. Emergency repairs, production stoppages, and collateral damage to adjacent components collectively account for a significant portion of operating expenditure in manufacturing environments. Traditional maintenance strategies address this risk poorly: reactive approaches accept the cost of sudden breakdowns, while time-based preventive schedules service equipment regardless of its actual condition, wasting resources and still missing faults that develop between inspection intervals.

The proliferation of low-cost IoT hardware has made a third approach viable for a much wider range of facilities. Inexpensive microcontrollers, open-source machine learning frameworks, and pay-as-you-go cloud services have matured to the point where a complete predictive maintenance pipeline no longer requires enterprise-grade infrastructure. Despite this, most commercially available platforms remain out of reach for smaller manufacturers. Systems such as **Siemens MindSphere**, **IBM Maximo**, and **GE Predix** are designed and priced for large industrial enterprises, bundling capabilities such as fleet-wide asset management, ERP integration, compliance reporting, and multi-site orchestration that most small and medium-sized facilities simply do not need. Licensing fees, mandatory professional services engagements, and the infrastructure required to run these platforms translate into capital commitments that are prohibitive for the small and medium-sized enterprises that make up a large share of industrial output in developing economies such as Pakistan. Rolling element bearings were selected as the target fault domain because they are present in virtually all rotating machinery, are among the leading causes of unplanned industrial stoppages, and produce vibration signatures that are well-suited to

learning-based analysis.

## 1.2 Problem Statement

Bearing degradation is a gradual process. In its earliest stages, the changes it produces in vibration amplitude and frequency content are small enough to be masked entirely by ambient noise. By the time a developing fault becomes perceptible through unusual sound or visible vibration, internal damage has typically progressed to the point where failure is imminent, often taking the shaft, housing, and adjacent components with it.

Conventional monitoring systems, including **SCADA** platforms with fixed-threshold alarms on individual sensor channels, are poorly suited to catching this early deterioration. These systems trigger alerts only when a measured variable crosses a predefined limit, meaning that the gradual, broadband changes characteristic of early-stage bearing wear pass entirely undetected until the damage is already severe. Periodic manual inspection suffers from the same fundamental limitation: faults that develop between inspection dates go unnoticed until the next scheduled check or until failure occurs first.

A further challenge is the scarcity of labeled fault data in real industrial settings. Machines spend the vast majority of their service life running normally, and genuine fault events are rare. Any system that depends on a large library of pre-labeled fault examples will therefore struggle in practice. What is needed is an approach that learns normal behavior from plentiful healthy data and flags deviations from it, reserving supervised classification for the confirmation stage once an anomaly has already been identified.

A further gap in the existing literature is the tendency to evaluate algorithms in isolation rather than as part of a deployable system. Most published work validates a detection or classification model against a benchmark dataset but stops short of demonstrating how that model would receive live sensor data, persist results, or surface alerts to an operator. This project addresses that gap by treating end-to-end integration as a first-class design

requirement rather than an afterthought.

## 1.3 Scope

This project covers the design, implementation, and validation of a complete IoT-based predictive maintenance system for rotating machinery, with rolling element bearings as the primary fault domain. The work spans four architectural layers: edge sensing, cloud data ingestion, machine learning inference, and frontend visualization.

The following are **within** the scope of this project:

- ESP32-based edge node acquiring tri-axial vibration, surface temperature, and motor current data, transmitting telemetry to Azure IoT Hub over a TLS-secured MQTT connection.
- Cloud ingestion pipeline using Azure IoT Hub, Azure Functions, and Azure Blob Storage.
- Two-stage machine learning pipeline comprising an LSTM Autoencoder for unsupervised anomaly detection and a CNN fault classifier for inner race, outer race, and rolling element fault identification.
- Containerized FastAPI inference service deployed on Azure Container Apps and a React-based real-time monitoring dashboard.
- System validation on the NASA IMS benchmark dataset, a custom developed dataset, and a live hardware test rig.

The following are explicitly **outside** the scope of this project:

- Remaining Useful Life (RUL) estimation, multi-machine monitoring, or integration with enterprise maintenance management systems.
- On-device machine learning inference; all inference runs server-side in the cloud.

- Fault detection for machinery without rolling element bearings.

## 1.4 Aims and Objectives

The **aim** of this project is to design and deploy a practical, low-cost IoT-based predictive maintenance system capable of detecting and classifying bearing faults through continuous real-time monitoring and machine learning, demonstrating that a complete deployable pipeline is achievable without enterprise-grade infrastructure.

To realize this aim, the following specific **objectives** are pursued:

1. Design and implement an ESP32-based edge sensing node that continuously acquires tri-axial vibration, surface temperature, and motor current data, and transmits timestamped telemetry to the cloud over a secured MQTT connection.
2. Build a cloud ingestion and storage pipeline on Microsoft Azure that reliably receives, validates, and persists raw sensor data for downstream consumption.
3. Develop an LSTM Autoencoder anomaly detection model and CNN fault classifier, forming a two-stage machine learning pipeline capable of early fault detection and fault type identification.
4. Deploy the machine learning pipeline as a containerized inference service and expose it through a React-based monitoring dashboard presenting health scores, fault alerts, and live telemetry in real time.
5. Validate system performance across benchmark, synthetic, and real-time hardware datasets to confirm practical early fault detection capability.

## 1.5 Outcomes

Completion of this project is expected to yield the following concrete outcomes:

- A functional edge-to-cloud data pipeline streaming authenticated sensor telemetry from an ESP32 node into Azure storage without manual intervention.
- A trained two-stage machine learning pipeline providing advance warning of bearing degradation and correct identification of fault type upon anomaly confirmation.
- A live containerized inference service and real-time React dashboard through which a maintenance engineer can monitor equipment health and act on fault alerts.
- A validated reference architecture demonstrating that a complete predictive maintenance system is achievable without proprietary industrial platforms, making it a viable option for resource-constrained manufacturers including SMEs in Pakistan.

## 1.6 Report Organization

The organization of the thesis is as follows:

**Chapter 2** reviews the background literature on bearing fault diagnosis and predictive maintenance, tracing the progression from physics-based models through traditional machine learning methods to modern deep learning frameworks, and identifying the gaps that motivate this work.

**Chapter 3** describes the hardware components and materials used in the construction of the edge sensing node, with component specifications and selection rationale provided for each.

**Chapter 4** details the system methodology across all four architectural layers — edge data acquisition, cloud ingestion, machine learning pipeline, and frontend visualization — covering design decisions, algorithms, and implementation specifics throughout.

**Chapter 5** presents the deployment and validation of the complete system, including containerization and cloud hosting of the inference backend, the Azure Functions API layer, the dashboard walkthrough, and performance evaluation results across three dataset

scenarios.

**Chapter 6** concludes the report with a summary of key contributions, a reflection on the limitations encountered, and recommendations for future work that could extend or build upon the proposed system.

# Chapter 2

## Background & Related Work

### 2.1 Bearing Fault Diagnosis: Fundamentals and Types

As previously mentioned, rolling element bearings are critical mechanical components in rotating machinery, and their failure can lead to severe operational disruptions, unplanned downtime, and costly maintenance. A typical rolling element bearing consists of four primary components: the inner race, outer race, rolling elements (balls or rollers), and a cage that maintains element spacing. Faults occurring in any of these components produce distinct vibration signatures that serve as the foundation for condition monitoring and fault diagnosis.

**Inner Race Faults** occur on the rotating inner ring surface due to fatigue, misalignment, or improper installation. Rolling elements repeatedly striking the defect generate periodic impulses at the Ball Pass Frequency Inner Race (BPFI):

$$\text{BPFI} = \frac{n}{2} f_r \left( 1 + \frac{d}{D} \cos \phi \right) \quad (2.1)$$

**Outer Race Faults** occur on the stationary outer ring due to localized stress or contamination. Because the defect position is fixed relative to the sensor, these faults produce

strong, consistent spectral peaks at the Ball Pass Frequency Outer Race (BPFO):

$$\text{BPFO} = \frac{n}{2} f_r \left( 1 - \frac{d}{D} \cos \phi \right) \quad (2.2)$$

**Rolling Element Faults** occur on the surface of the rolling elements themselves, generating complex vibration patterns at the Ball Spin Frequency (BSF):

$$\text{BSF} = \frac{D}{2d} f_r \left( 1 - \left( \frac{d}{D} \cos \phi \right)^2 \right) \quad (2.3)$$

In all three equations,  $n$  is the number of rolling elements,  $f_r$  is the shaft rotational frequency,  $d$  is the rolling element diameter,  $D$  is the pitch diameter, and  $\phi$  is the contact angle. These characteristic frequencies enable fault localization and underpin both traditional signal processing and modern learning-based diagnostic systems.

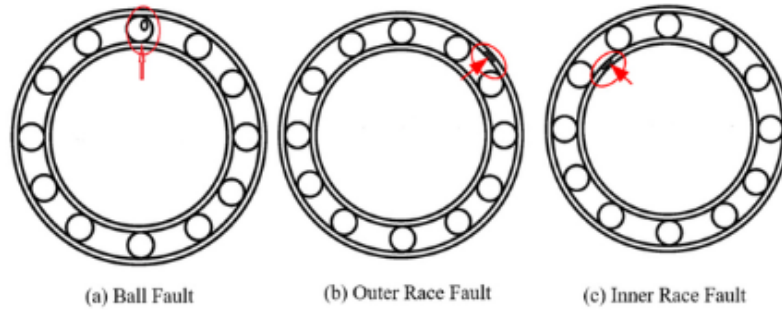


Figure 2.1: Schematic representation of bearing fault types: (a) Inner race fault, (b) Outer race fault, (c) Rolling element fault

## 2.2 Related Work

Bearing fault diagnosis has evolved through three broad stages: physics-based modeling, traditional machine learning, and deep learning. Each stage addressed limitations of the previous, and understanding this progression motivates the design choices made in this project.

### **2.2.1 Physics-Based and Signal Processing Approaches**

Initial condition monitoring was based on the fault frequency characteristics outlined above, in combination with traditional signal processing methods, including envelope analysis and spectral kurtosis. These concepts were comprehensively addressed by Randall and Antoni [1], with envelope analysis established as the standard baseline for bearing diagnostics. Spectral kurtosis was further explored by Antoni [2] as a means of identifying impulsive signatures within the noise background, while its application in detection of bearing faults under varying loads was demonstrated by Immovilli et al. [3]. While highly insightful, these techniques operate under ideal conditions, require accurate geometric specifications of bearings, and fail under noisy and variable loading conditions. The lifetime of a bearing is modeled using the Lundberg–Palmgren and ISO 281 standards to calculate the bearing life cycle using load and material parameters; however, these models do not consider any sensor information and hence cannot adapt to actual machine operation conditions. Caesarendra and Tjahjowidodo [4] presented an in-depth review of various signal processing methods used in bearing condition monitoring at low speeds.

### **2.2.2 Traditional Machine Learning**

To overcome the rigidity of physics-based models, researchers combined handcrafted time and frequency domain features, such as RMS, kurtosis, and spectral peaks, with classical classifiers including Support Vector Machines, k-Nearest Neighbors, and Random Forests. Widodo and Yang [5] demonstrated the effectiveness of SVMs for fault classification, while Cerrada et al. [6] showed that ensemble methods improve robustness in noisy environments. Lei et al. [7] further surveyed the application of machine learning to machinery health monitoring, highlighting the breadth of feature-classifier combinations explored in the literature. Konar and Chattopadhyay [8] applied k-Nearest Neighbor classifiers to bearing fault diagnosis using vibration features, achieving strong classifica-

tion accuracy on controlled test rig data. Uddin and Islam [9] demonstrated that Random Forest classifiers with carefully selected time-domain features offer competitive performance with lower computational overhead than deep models. The central limitation of these approaches is their dependence on manual feature engineering, which requires significant domain expertise and often fails to capture the subtle signal changes associated with early-stage faults.

### **2.2.3 Deep Learning for Fault Classification**

Convolutional Neural Networks (CNNs) addressed the feature engineering bottleneck by learning hierarchical representations directly from raw or minimally processed vibration signals. Wen et al. [10] proposed a CNN-based fault diagnosis model eliminating hand-crafted features, while Zhang et al. [11] demonstrated strong noise robustness using deep CNN architectures. Guo et al. [12] proposed a hierarchical adaptive deep convolution neural network and showed that deeper architectures capture more discriminative fault features from raw signals. Time–frequency representations such as Short-Time Fourier Transform (STFT) and wavelet transforms have also been used to convert 1D signals into structured 2D inputs, further improving CNN classification accuracy. Chen et al. [13] combined attention mechanisms with CNNs to selectively focus on fault-relevant frequency components, improving classification under variable operating conditions. Recurrent architectures have also been explored: Zhang et al. [14] applied LSTM networks directly to raw vibration sequences for fault classification, exploiting temporal dependencies that CNNs applied to fixed windows cannot capture. Zhao et al. [15] provided a systematic review of deep learning methods for rotating machinery fault diagnosis, confirming CNNs as the dominant architecture across benchmark evaluations. Despite strong benchmark performance, all of these supervised approaches require large labeled datasets, which are rarely available in practice because industrial machinery operates under healthy conditions the vast majority of the time.

## 2.2.4 Unsupervised Anomaly Detection

To address the labeled data scarcity problem, unsupervised methods have gained considerable attention. Autoencoder-based models, particularly LSTM Autoencoders, learn a compact representation of normal operating behavior and detect anomalies through elevated reconstruction error. Malhotra et al. [16] demonstrated that LSTM encoder-decoder architectures can effectively capture temporal dependencies in sensor signals and flag deviations from normal patterns without any fault labels. Chalapathy and Chawla [17] further surveyed deep learning approaches to anomaly detection, establishing reconstruction-based methods as the most practical solution when labeled fault data is unavailable. Principi et al. [18] validated autoencoder-based anomaly detection specifically for rotating machinery, showing that reconstruction error thresholds derived from normal training data generalize well to unseen fault conditions. Park et al. [19] extended the autoencoder paradigm to multimodal sensor fusion, demonstrating improved anomaly detection sensitivity when vibration, temperature, and current signals are jointly encoded. Munir et al. [20] proposed a CNN-based anomaly detection framework for time series that complements reconstruction-based approaches by predicting future signal values and flagging unpredictable deviations, providing an alternative detection strategy that does not require an encoder-decoder architecture. This paradigm is especially suited to early fault detection, where the signal changes are subtle and no labeled examples exist.

## 2.3 Gaps in Existing Research

Three limitations in the existing literature directly motivate this project. First, the majority of published work relies on supervised classification, which presupposes the availability of labeled fault data that is rarely obtainable in real industrial settings. Second, established commercial platforms such as **Siemens MindSphere**, **IBM Maximo**, and **GE Predix** are priced and architected for large enterprises, placing them out of reach for small and medium-sized manufacturers. Third, most academic studies evaluate algorithms in isola-

tion against benchmark datasets without demonstrating how a model would receive live sensor data, persist results, or surface alerts to an operator. This project treats end-to-end integration as a first-class design requirement, addressing all three gaps within a single deployable architecture.

# Chapter 3

## Material & Component

### 3.1 Device Hardware Components

The predictive maintenance system we developed has a small edge device that gets machine health data and sends it to the cloud for analysis. This device has a microcontroller, sensors that are used in industries, special connectors and a protective case to make sure it works well and is easy to change parts.

#### 3.1.1 Central Processing Unit

The central processing unit of the device uses the ESP32-WROOM-32 microcontroller module. This module is the control unit that gets data from sensors, processes signals, and sends information to the cloud. The ESP32-WROOM-32 is often used in Internet of Things applications because it has built-in communication features and strong processing capability.

We chose this module for the following reasons:

- **Low Cost and Market Availability:** The ESP32-WROOM-32 is easily available in international electronics markets at a low cost compared to many industrial-

grade microcontrollers. This makes the device affordable for large-scale deployment across machines, which is important in predictive maintenance systems.

- **Wi-Fi Capability:** The ESP32 has built-in Wi-Fi functionality, which we use to send real-time sensor data to cloud-based platforms. This eliminates the need for additional communication modules, reducing system complexity, wiring requirements, and overall hardware cost.
- **High Processing Capability:** The module has a dual-core Tensilica Xtensa LX6 processor that operates at frequencies up to 240 MHz. This enables handling of multiple operations simultaneously, including sensor data acquisition, filtering, local preprocessing, and wireless data transmission.
- **Multiple Peripheral Interfaces:** The ESP32 supports communication interfaces such as I2C, SPI, UART, PWM, and Analog-to-Digital Converter channels. These interfaces allow integration with many sensors, enabling flexible system expansion and reliable data communication between components.
- **Low Power Consumption:** The microcontroller supports power-saving modes, allowing the device to operate efficiently during continuous monitoring applications.
- **Extensive Software Support:** The ESP32 platform is supported by documentation, open-source libraries, and a large developer community.

Due to these combined technical advantages, the ESP32-WROOM-32 serves as a reliable and efficient central processing unit for the predictive maintenance device, enabling real-time data acquisition, processing, and cloud communication.



Figure 3.1: ESP32 WROOM-32 Microcontroller

### 3.1.2 Sensors

Our predictive maintenance device has sensors to monitor key machine health parameters, including vibration, current consumption, and temperature. These parameters are indicators of machine condition and enable early detection of mechanical and electrical faults.

#### 3.1.2.1 ADXL345 Accelerometer (Vibration Sensor)

The ADXL345 triaxial accelerometer measures vibration signals generated from the bearings of the rotating machine. The selection of this sensor was based on its ability to measure acceleration along three axes (X, Y, and Z), enabling comprehensive monitoring of vibration behavior in multiple directions. Its digital I2C/SPI communication capability allows reliable integration with the ESP32 microcontroller, while its high sensitivity enables detection of small vibration variations associated with bearing faults, imbalance, and misalignment. The sensor is positioned near the bearing region to capture accurate vibration data during machine operation.

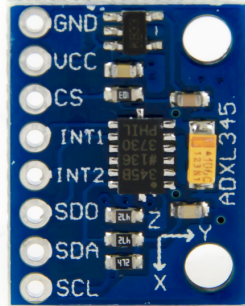


Figure 3.2: ADXL345 Accelerometer (Vibration Sensor)

### 3.1.2.2 SCT-013 Current Sensor

The SCT-013 current transformer sensor monitors the input current supplied to the motor. This sensor was selected due to its non-invasive measurement capability, allowing current monitoring without direct electrical contact with the conductor, which improves installation safety and ease of deployment. Monitoring current consumption helps detect abnormal load conditions, mechanical resistance, and potential electrical faults in the motor. The sensor provides reliable AC current measurement and enables continuous monitoring of motor operating conditions.



Figure 3.3: SCT-013 30A/1V non-invasive current sensor

### 3.1.2.3 MAX6675 with K-Type Thermocouple (Temperature Sensor)

The temperature monitoring system consists of a MAX6675 interface module paired with a K-type thermocouple to measure temperature variations in the motor or bearing region. This combination was selected due to its wide temperature measurement range and reliable digital output through SPI communication, enabling accurate temperature acquisition by the ESP32. Temperature monitoring plays a significant role in predictive maintenance, as abnormal temperature rise may indicate friction, lubrication failure, or motor overload conditions. The K-type thermocouple provides stable performance in industrial environments, making it suitable for continuous monitoring applications.

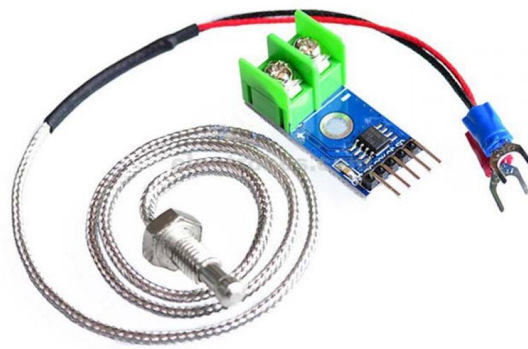


Figure 3.4: MAX6675 Module and K-type Thermocouple

### 3.1.3 Sensor Connectivity

All sensors in the maintenance device are connected to the central ESP32 unit using JST connectors. The use of JST connectors enables a plug-and-play architecture, allowing sensors to be easily connected, replaced, or serviced without complex rewiring. This improves installation efficiency and reduces the likelihood of wiring errors during deployment. Additionally, JST connectors provide secure and reliable electrical connections, which are essential for maintaining consistent signal transmission from sensors to the mi-

crocontroller. The modular nature of this connectivity approach enhances maintainability and supports scalability of the system for future sensor integration.

### 3.1.4 Device Enclosure

The complete device is enclosed within a custom-designed 3D printed casing, which houses the internal circuitry mounted on a printed circuit board along with the ESP32 microcontroller. The casing is designed with ports that allow sensors to be connected externally using JST connectors, enabling a modular plug-and-play configuration. Additionally, the enclosure provides structured mounting for the PCB and wiring, resulting in a compact and organized device assembly. The use of a dedicated casing demonstrates that the system extends beyond a basic prototype and moves toward a practical and deployable hardware solution.

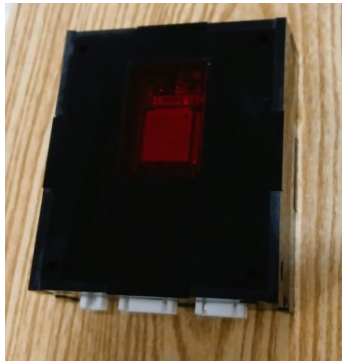


Figure 3.5: Device in 3D-printed casing



Figure 3.6: Device in 3D-printed casing

## 3.2 Software

### Cloud & ML Platforms:

- Microsoft Azure (IoT Hub, ML, Storage Accounts, Container Apps)
- Google Colab

### Development & Simulation:

- Arduino IDE

- Proteus
- MATLAB

#### **Deployment & APIs:**

- Docker
- FastAPI

### **3.3 Data Collection**

This project draws on three distinct datasets: the NASA IMS benchmark dataset, the Case Western Reserve University (CWRU) bearing dataset, and a custom artificially degraded dataset developed on the test rig. Each serves a different purpose in the validation pipeline and is described in the subsections below.

#### **3.3.1 Test Rig Description**

To evaluate the predictive maintenance system under controlled conditions, a mechanical simulator test rig was constructed to replicate the operational characteristics of rotating machinery. The rig consists of a single-phase electric motor driving an extended shaft assembly, onto which a **SKF 6205-2RS** deep groove ball bearing is mounted as the primary test component. A 2 kg load disk is fitted at the end of the shaft to introduce mechanical load and generate measurable variations in vibration and current consumption, simulating realistic industrial operating conditions.

The original intention was to run the rig continuously until bearing failure, mirroring the run-to-failure philosophy of the NASA IMS dataset, so that the anomaly detection model could be evaluated on how far in advance it flagged degradation relative to actual failure. However, as discussed in Section 3.3.2, replicating those conditions was not feasible in a supervised academic setting. The test rig was therefore repurposed for controlled fault

injection experiments, described in Section 3.3.3.



(a) Inner Race Fault



(b) Outer Race Fault



(c) Rolling Element Fault

Figure 3.7: Bearing Fault Conditions Used in Experiments

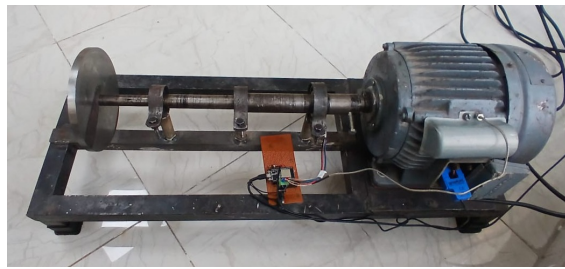


Figure 3.8: Test Rig Assembly

### 3.3.2 NASA IMS Bearing Dataset

The NASA Intelligent Maintenance Systems (IMS) bearing dataset [21] is one of the most widely used benchmarks for bearing prognostics research. The dataset was collected from a purpose-built test rig consisting of four bearings mounted on a single rotating shaft, loaded with a 600 lb radial force and driven at a constant speed of 2000 RPM. Vibration data was recorded at 20 kHz using accelerometers mounted on the bearing housing, and each test was run continuously until at least one bearing reached failure, yielding three complete run-to-failure datasets (Set 1, Set 2, and Set 3) spanning periods of days to weeks.

The run-to-failure structure of this dataset makes it particularly well suited for evaluating early fault detection: because the exact point of failure is known, it is possible to measure precisely how many hours or days in advance an anomaly detection model raises an alert.

This characteristic made it the reference benchmark for validating the LSTM Autoencoder developed in this project.

The original plan for the test rig was to replicate this run-to-failure paradigm: run the rig continuously under load until the mounted bearing degraded to failure, and use the resulting data to evaluate how early the system could detect the onset of degradation. This was not achievable in practice. Leaving an unsupervised motorized rig running under continuous mechanical load for potentially several weeks posed unacceptable safety and supervision risks in an academic laboratory environment. On the advice of the project supervisor, an alternative approach using an artificially constructed degradation dataset was adopted instead, as described in the following section.

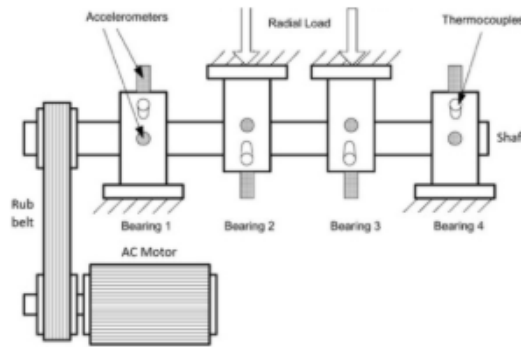


Figure 3.9: Simulator used for NASA run-to-failure experiment

### 3.3.3 CWRU Dataset and Artificial Degradation Dataset

#### 3.3.3.1 CWRU Bearing Dataset

The Case Western Reserve University (CWRU) bearing dataset [22] is the standard reference for bearing fault classification research. It was collected from a test rig comprising a motor, torque transducer, and dynamometer, with accelerometers mounted at the drive end and fan end of the motor housing. Faults were seeded into bearings of the same model used in this project using **Electrical Discharge Machining (EDM)**, producing precisely controlled defects at three locations: the inner race, outer race, and rolling elements,

across multiple fault diameters. The dataset is widely used to benchmark fault classifiers due to its clean, well-labeled structure and the availability of data across multiple load conditions.

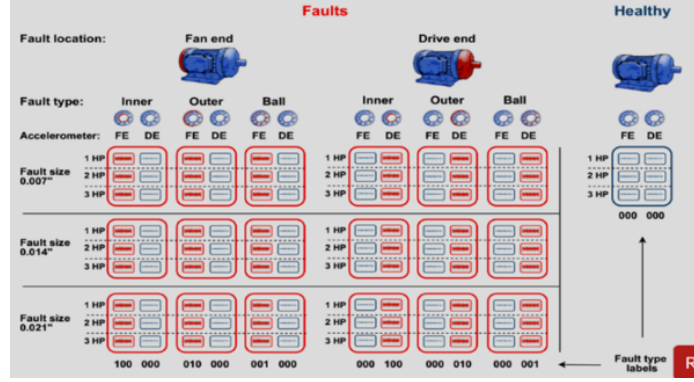


Figure 3.10: Detailed structure of CWRU dataset

### 3.3.3.2 Artificial Degradation Dataset

Since access to EDM equipment was not available, faults were introduced into the test rig bearing using **mechanical grinding**, producing surface defects of approximately 0.1–0.3 mm in width. While these faults are visibly very small, they produce measurable changes in vibration characteristics once the bearing is installed and the rig is running, with noticeable acoustic and vibrational noise, mapping them to failure instead of micro-defects. The bearing under consideration in our experiment was the largest-diameter bearing on the test rig. Faulty or healthy bearings were installed at this position as needed, and all data was collected at a sampling rate of 1 kHz.

Three fault types were introduced, aligned with the CWRU fault categories: **inner race faults**, **outer race faults**, and **rolling element faults**. For inner and outer race conditions, two severity levels were created (low and high), while a single severity level was used for rolling element faults. This yielded the following experimental conditions:

| Type    | Low Severity    |            |            | High Severity |            |
|---------|-----------------|------------|------------|---------------|------------|
| No Load | Rolling Element | Outer Race | Inner Race | Outer Race    | Inner Race |
| Load    | Rolling Element | Outer Race | Inner Race | Outer Race    | Inner Race |

Figure 3.11: Data Collection Conditions

Vibration data was collected for each condition independently. These segments were then **concatenated in order of increasing severity** to form a synthetic degradation sequence: normal operation followed by low-severity fault data followed by high-severity fault data. This artificially constructed progression was treated as a proxy for gradual bearing degradation, acknowledging that it does not replicate the continuous physical wear process of a true run-to-failure experiment.

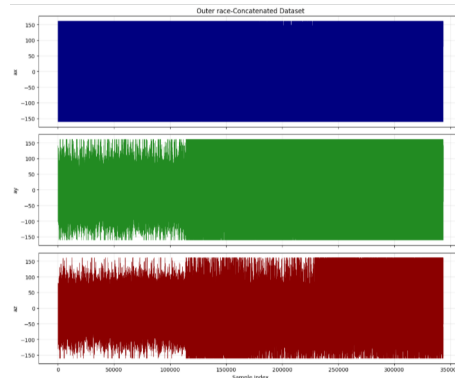


Figure 3.12: Sample Concatenated Data in order (normal vibration data-low severity outer race fault vibration data-high severity outer race fault vibration data)

### 3.3.4 Dataset Validation Strategy

Following supervisor guidance, an artificially concatenated dataset was adopted as a proof-of-concept degradation signal for validating the anomaly detection pipeline. The artificial dataset was aligned to the NASA IMS dataset using **Dynamic Time Warping (DTW)**

to enable temporal mapping between the artificial degradation trajectory and the known run-to-failure progression. This alignment allowed an estimate of how early the LSTM Autoencoder would have flagged degradation onset relative to the failure point in the IMS dataset.

Additionally, the same LSTM Autoencoder was also independently tested on the NASA IMS dataset. This gave us a look at how well the LSTM Autoencoder model works in general and showed that it performs well on a real run-to-failure dataset. Both validation results are reported in Chapter 5.

# Chapter 4

## Methodology

### 4.1 System Overview

The proposed IoT-based Predictive Maintenance System follows an end-to-end architecture spanning four layers: edge data acquisition, cloud ingestion, machine learning inference, and frontend visualization. An ESP32 microcontroller equipped with vibration, temperature, and current sensors transmits timestamped telemetry over a secure MQTT connection to Azure IoT Hub, where an Event Hub-triggered Azure Function processes and persists it in Azure Blob Storage alongside pre-trained model artifacts. A containerized FastAPI inference service on Azure Container Apps then executes the two-stage machine learning pipeline, in which the LSTM Autoencoder flags anomalies and the 1-D CNN fault classifier identifies the fault type upon confirmation. Results are surfaced to the operator through a React monitoring dashboard presenting live telemetry, health scores, and degradation trends in real time.

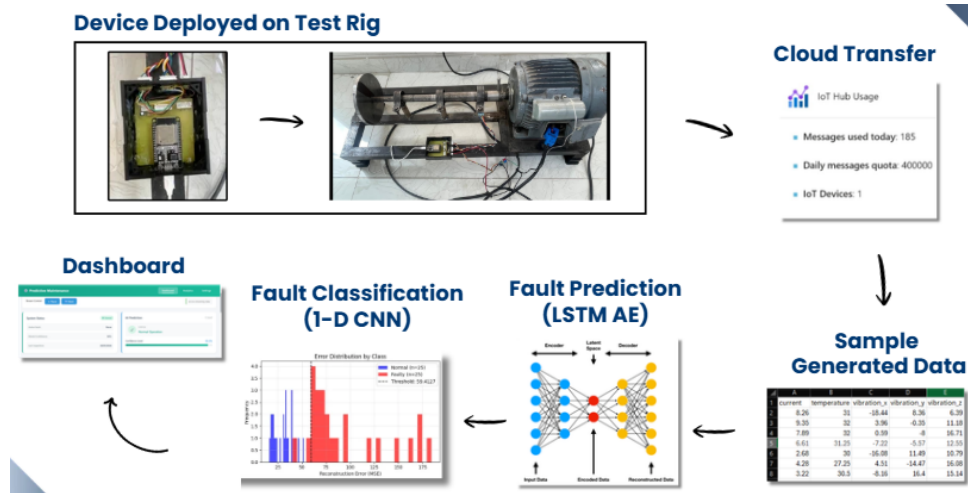


Figure 4.1: System-Level Overview: From Edge Device Deployment to Dashboard Visualization

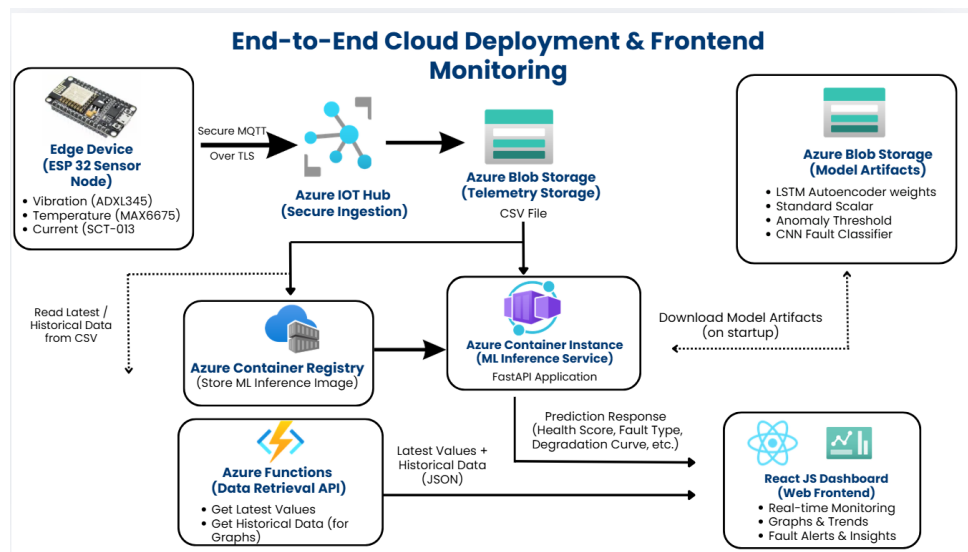


Figure 4.2: End-to-End Cloud Deployment Architecture of the IoT-Based Predictive Maintenance System

## 4.2 Edge Data Acquisition Layer

This layer acts as a bridge between the physical machine and the cloud environment. It is deployed in an ESP32 microcontroller board that uses three sensors to measure the mechanical and electrical parameters of the observed machine. The collected sensor data is then sent in the form of telemetry packets to the cloud in fixed intervals.

### **4.2.1 ESP32 Microcontroller**

This project implements the telemetry functionality using an ESP32 edge node—a cheap, low-power system-on-chip created by Espressif Systems. In particular, ESP32 uses a dual-core processor (Xtensa LX6) running up to 240 MHz, has 520 KB of on-board SRAM storage, 4 MB of internal Flash memory, and integrates Wi-Fi (802.11 b/g/n) and Bluetooth 4.2 wireless technologies. It comes equipped with I2C, SPI, UART interface connectivity, as well as 12-bit ADC input pins and thus can communicate with up to twelve different sensors simultaneously. This combination of computational resources, wireless connectivity, and low energy consumption makes it a perfect candidate for embedded sensing tasks that would require simultaneous local and remote data transfer without deploying an embedded Linux OS.

In this setup, the edge telemetry node implemented using ESP32 chip communicates with all sensors and performs computations of any derived variables, such as RMS current, builds JSON messages and timestamps them before establishing an authenticated connection to Azure IoT Hub via MQTT protocol. When starting, the edge node connects to the Wi-Fi network, establishes a connection with an NTP server to set its real-time clock correctly, generates a shared access signature, and establishes an authenticated connection to the MQTT broker.

### **4.2.2 Sensor Configuration & Signal Acquisition**

A total of three sensors are used for interfacing with the ESP32 for the purpose of acquisition of physical signals related to machine health monitoring. ADXL345 Tri-axial accelerometer sensor interfaced through I2C communication protocol senses vibration acceleration along x, y, and z directions in the defined range of  $\pm 16g$ . Vibration is the key parameter that is being utilized as the diagnostic signal in this case because mechanical degradation of the machine, especially the bearing fault in rotating machinery occurs as vibration parameter change on all three axes. Surface temperature of the machine is

sensed through K-type thermocouple connected through MAX6675 SPI module and output is measured in degree centigrade. Current flowing into the motor is measured using SCT-013 current transformer by measuring voltage across ESP32 ADC pin. RMS value of the current can be measured using RMS voltage measurement formula where first of all 2000 samples are acquired in 12 bit resolution with 11 dB attenuation. After removing DC component using 200 samples averaging, mean square of samples are calculated and converted into RMS value of current using CT ratio. Although all three data points are collected and sent to the cloud,<sup>13</sup> the inference pipeline based on machine learning works only with the vibration signal, since the current and temperature parameters were constant within the operating range that was tested in this project; thus, vibration became the most discriminating signal for faults. A JSON message, including the time synchronized via the NTP protocol in format HH:MM:SS, triaxial accelerations  $a_x$ ,  $a_y$ ,  $a_z$  measured in  $m/s^2$ , the root-mean-square of the motor current value in amps, temperature on the surface in degrees Celsius, and a ground truth label in binary form used in fault injection experiments is generated per each sensing cycle. The telemetry messages are broadcast at intervals of 200 ms.

### **4.2.3 Secure MQTT Transmission**

MQTT protocol, a lightweight pub-sub messaging protocol that allows for persistent sessions with minimal packet overhead is used for communication between the ESP32 and Azure cloud. The MQTT connection is secured using the SSL/TLS protocol, and it is made on the 8883 port using the WiFiClientSecure library that encrypts all telemetry data sent to Azure IoT Hub in transit.

Authentication is performed by using SAS tokens that are generated locally on the device itself. The SAS token is constructed from the base-64 decoded symmetric key of the device using the HMAC-SHA256 hashing function with a string formed by the IoT Hub resource URI concatenated with the expiry time value in Unix format. The token is

generated by converting the signature bytes into a Base64 and URL-encoding them. The resulting SAS token is sent as the password for connecting the device to the Azure IoT Hub in order to provide it with temporary access. Each telemetry message is published to the specific device MQTT topic

## **4.3 Cloud Ingestion Pipeline**

Once the telemetry is sent out by the ESP32 edge node, it gets consumed by the cloud ingestion pipeline that ingests the telemetry, processes it, and stores it persistently in a format that can be used to drive machine learning inference. The cloud ingestion pipeline comprises of three tightly-coupled Azure services namely, Azure IoT Hub, Azure Function triggered by an Event Hub, and Azure Blob Storage.

### **4.3.1 Azure IoT Hub**

The Azure IoT Hub provides a cloud-based interface used to receive messages from the devices' telemetry. Specifically, it is an IoT Hub designed to enable scalable two-way communication between edge devices and the cloud by implementing device authentication using SAS tokens as discussed in section 4.2.3. Every JSON message that the ESP32 publishes gets received at the IoT Hub, and it is then redirected to its inbuilt Event Hub-compliant endpoint. This makes it possible for the message stream that reaches IoT Hub to be accessed by downstream processing services. The availability of IoT Hub means that the development team is not required to manage any servers.

### **4.3.2 Event Hub-Triggered Azure Function**

The receipt of each telemetry message at the IoT Hub's Event Hub compatible endpoint causes an Azure Function to be fired up, where the function has been defined to have an Event Hub input binding. An Azure Function is a type of serverless compute platform where discrete blocks of code run in reaction to events rather than running servers con-

stantly. In this case, the Azure Function fired will be used to parse the JSON payload data from the message, verify that all the required sensor fields are present, and retrieve the columns of interest, which include: timestamp, triaxial vibrations acceleration, motor current, surface temperature, and ground truth label. 15 These record columns are added to a CSV file that resides on the Azure Blob Storage account.

### **4.3.3 Azure Blob Storage**

Azure Blob Storage is utilized as the database for the system, consisting of two different containers. The first one stores the telemetry CSV file, which consists of all the data that the edge device has provided to the system in form of telemetries. In this file each row represents a specific telemetry, and the data consists of the following: timestamp (HH:MM:SS), vibration accelerations on the X, Y, and Z axes ( $\text{m/s}^2$ ), motor current (A), surface temperature ( $^{\circ}\text{C}$ ), and ground truth label (boolean). The second container acts as the place for storage of all the artifacts, such as the pre-trained LSTM Autoencoder weights, StandardScaler object, the anomaly detection threshold, and the CNN weights of the fault classifier. All of those artifacts need to be uploaded only once after the training process and then downloaded by the inference service before starting the prediction process.

## **4.4 Machine Learning Pipeline**

The machine learning pipeline constitutes the main analysis module within the proposed predictive maintenance framework. Its architecture involves a hybrid approach combining unsupervised anomaly detection via an LSTM Autoencoder and supervised classification via a 1-D Convolutional Neural Network (CNN). The process takes as input vibration signals acquired by the edge layer and provides as output both fault detection and type classification.

#### 4.4.1 Data Preprocessing

Raw vibration data recorded from the ESP32 sensor are first cleaned and preprocessed. Missing and invalid values in the tri-axial vibration signals ( $a_x$ ,  $a_y$ ,  $a_z$ ) are deleted, while timestamps are validated. Outliers are discarded and the remaining data transformed into numeric vectors to enable passing valid vibration signals to the feature extraction step.

#### 4.4.2 Feature Extraction and Windowing

A sliding window of a predefined length (1024 samples) and shift (512 samples) is applied to the vibration signal. A set of time-domain and frequency-domain features is extracted from the signal at every window across all three axes. Time-domain features include mean, standard deviation, RMS, peak, peak-to-peak, skewness, and kurtosis. Frequency-domain features include spectral centroid and dominant frequency calculated via Fast Fourier Transform (FFT). Each window generates a feature vector of 27 dimensions (9 features  $\times$  3 axes).

#### 4.4.3 Normalization and Sequence Creation

The obtained feature vectors are normalized through MinMax normalization, which is fitted only on normal operational data, avoiding any data leakage due to fault scenarios into the normalization process. In order to capture temporal dependencies, the normalized feature vectors are segmented into sequences of size 10 through a sliding window approach, where each sequence corresponds to a brief temporal evolution of machine behavior and acts as the input for the LSTM Autoencoder.

#### 4.4.4 LSTM Autoencoder for Anomaly Detection

An LSTM Autoencoder is employed to learn the normal operating behavior of the system. The model is chosen specifically because it trains exclusively on healthy data, requiring

no fault labels, and its recurrent architecture captures the temporal context of vibration patterns across consecutive windows rather than treating each window in isolation. This makes it well-suited to detecting the gradual, time-correlated changes that characterize early-stage bearing degradation.

The model follows an encoder-decoder architecture. The encoder consists of two stacked LSTM layers (32 units and 8 units) that compress the input sequence into a compact latent representation. The decoder mirrors this structure symmetrically, reconstructing the original input sequence from the latent space, with a TimeDistributed dense output layer producing the final reconstruction.

During inference, anomaly detection is performed using the reconstruction error, computed as the Mean Absolute Error (MAE) between the input and reconstructed sequence:

$$\text{Error} = \frac{1}{N} \sum |X_{\text{input}} - X_{\text{reconstructed}}| \quad (4.1)$$

The detection threshold is set as the mean reconstruction error on the normal training data plus three standard deviations:

$$\theta = \mu_{\text{train}} + 3\sigma_{\text{train}} \quad (4.2)$$

This threshold is derived statistically from the training distribution rather than set arbitrarily, ensuring that it reflects the natural variability of healthy operating conditions. Because the model has only ever learned to reconstruct normal behavior, sequences containing fault-related vibration patterns produce significantly elevated reconstruction errors that exceed this threshold, flagging them as anomalous. Sequences within the healthy operating range are reconstructed with low error and classified as normal.

The anomaly detection performance is evaluated using F1-score computed separately for the normal and fault classes. The **fault detection F1-score** ( $F1_{\text{fault}}$ ) measures the model's

ability to correctly identify anomalous sequences, while the **normal F1-score** ( $F1_{\text{normal}}$ ) measures correct identification of healthy states. Together, these provide a balanced evaluation that accounts for both missed detections and false alarms. The primary metric of interest for the predictive maintenance objective is the **advance warning time**: the duration between the confirmed fault detection timestamp  $t_2$  and the known failure point, which quantifies how early the system raises an alert relative to actual failure.

#### 4.4.5 K-Consecutive Fault Detection Mechanism

In order to decrease false alarms and increase the accuracy of detection, the approach of K-consecutive faults is applied. Thus, instead of identifying the fault due to anomaly in one sequence, it must be identified through  $K = 10$  consecutive anomalies with reconstruction error above the threshold level. Two timestamps are calculated:

- $t_1$  – index of the first detected anomalous sequence;
- $t_2$  – index where  $K$  consecutive anomalies are identified.

Thus, the duration between  $t_1$  and  $t_2$  is called the lead time of early warning window, while the distance between  $t_2$  and failure point indicates the advance warning for the maintenance engineer.

#### 4.4.6 CNN-Based Fault Classification

When the fault is identified using anomaly detection module, it is classified using 1D CNN directly from the vibration signals of window size 50 data points. It consists of two 1-D convolutional layers with ReLU activation function, max-pooling followed by fully connected layers for feature extraction to the three possible fault classes: inner race fault, outer race fault, and rolling element fault. The predicted class is selected based on majority voting of all classifications performed.

#### 4.4.7 Validation Strategy

Machine learning model validation involved three stages of varying complexity levels, starting with a custom-made dataset to real-time hardware validation. Each stage employed a different methodology in preparing the inputs for the 27-dimensional feature vector that the LSTM-Autoencoder model requires.

##### 4.4.7.1 Phase 1: Custom Concatenated Dataset

The first validation phase used the artificially constructed degradation dataset described in Section 3.3.3. Tri-axial vibration data ( $a_x, a_y, a_z$ ) was collected directly from the ESP32 accelerometer mounted on the test rig under three conditions: normal operation, low-severity fault, and high-severity fault. These segments were concatenated in order of increasing severity to simulate a progressive degradation trajectory.

Each segment was processed through the standard windowing and feature extraction pipeline: windows of 1024 samples with a stride of 512 were applied, and 9 features were extracted per axis, yielding a 27-dimensional feature vector per window. The MinMax scaler was fitted exclusively on the normal segment, and the full concatenated sequence was then scaled and grouped into sequences of length 10 before being passed to the LSTM Autoencoder. The model was trained on the normal segment only, and the threshold  $\theta = \mu_{\text{train}} + 3\sigma_{\text{train}}$  was computed from the training reconstruction errors.

Because this dataset does not have a true run-to-failure endpoint, the reconstruction error profile from the custom dataset was aligned to the NASA IMS run-to-failure timeline using **Dynamic Time Warping (DTW)**. A window of  $\pm 20$  sequences around the confirmed fault index  $t_2$  was extracted from the normalized custom error profile, and this pattern was matched against the normalized NASA IMS error series by minimizing the DTW distance:

$$\text{DTW}(s_1, s_2) = \min_{\text{path}} \sum_{(i,j) \in \text{path}} |s_1(i) - s_2(j)| \quad (4.3)$$

The best-matching position in the NASA timeline was identified, and the time remaining from that position to the end of the NASA run (corresponding to bearing failure) was computed as an estimate of the advance warning the system would have provided under real degradation conditions.

#### 4.4.7.2 Phase 2: NASA IMS Benchmark Dataset

The second phase validated the model directly on the NASA IMS run-to-failure dataset, providing an independent benchmark evaluation using real degradation data. Three bearing fault scenarios were evaluated: Set 1 Bearing 3 (inner race defect), Set 1 Bearing 4 (rolling element defect), and Set 2 Bearing 1 (outer race defect).

The NASA IMS dataset presents a specific data preparation challenge: each time-stamped file contains raw vibration readings sampled at 20 kHz, but the number of available channels and their interpretation differs across dataset sets. In Set 1, each bearing is represented by two channels (for example, Bearing 3 uses columns 4 and 5, and Bearing 4 uses columns 6 and 7), providing two physical measurement axes. In Set 2 and Set 3, each bearing is represented by a single channel. To maintain consistency with the 27-dimensional feature vector expected by the LSTM Autoencoder, a third synthetic axis  $z$  was constructed as the absolute difference between the two available channels:

$$z = |x - y| \quad (4.4)$$

For single-channel bearings in Set 2, the available channel was assigned to both  $x$  and  $y$ , and  $z$  was set to zero. This approach preserves the three-axis feature extraction pipeline without introducing separate model variants for different dataset structures.

For each bearing, the model was trained on the first 25% of the run-to-failure sequence, representing the period of healthy operation at the start of each test. The scaler was fitted on this training portion, and the threshold  $\theta = \mu_{\text{train}} + 3\sigma_{\text{train}}$  was computed independently for each bearing from its own training reconstruction errors. The model was then evaluated across the full sequence, and advance warning time was measured as the duration from the confirmed fault timestamp  $t_2$  to the final recorded timestamp, which corresponds to the point of bearing failure.

#### 4.4.7.3 Phase 3: Real-Time Hardware Validation

This phase evaluated the end-to-end deployment process using live data from the ESP32 node that is physically connected to the test rig. Every 200 milliseconds, ESP32 sends out tri-axial vibration signals  $a_x$ ,  $a_y$  and  $a_z$  to Azure services, via the entire data ingestion process discussed in section 4.3, and accumulate the values into a telemetry CSV file in Azure Blob Storage.

During inference, the `/predict` endpoint loads the accumulated telemetry data, applies exactly the same preprocessing (1024 samples with 512 strides), feature extraction, Min-Max standardization, and sequence creation (sequence length of 10) as discussed in the previous phases, and directly input the resultant  $N \times 10 \times 27$  sequences into LSTM Autoencoder. On fault detection by the K consecutive method, raw vibration data are passed into the CNN classifier to classify the fault type. The results were delivered to the React web dashboard instantly to validate the pipeline accuracy with real sensor data and real cloud environment.

#### 4.4.8 Final Output Generation

The output consists of many details, including the overall system status (normal or fault), the identified fault class, the reconstruction error over time, fault detection times  $t_1$ ,  $t_2$ , F1 scores for each class, and advance warning time estimate. This output is forwarded to

the FastAPI inference service and subsequently rendered on the frontend dashboard for operator decision support.

## **4.5 Serverless API Layer**

In order to connect telemetry data in the form of persisted telemetry data in Azure Blob Storage to the frontend monitoring dashboard, a separate API layer is created using serverless architecture in Azure Functions. The fact that we have separated the responsibilities of writing incoming data to Azure Blob Storage from providing data to the frontend through API using serverless architecture is deliberate.

## **4.6 Frontend Architecture**

The frontend is implemented using React's single-page application, which is intended for real-time monitoring purposes. Since the frontend utilizes a modular design, it allows the updating of individual components within the dashboard without a complete refresh of the entire page.

### **4.6.1 API Integration & Data Sources**

There are two backend services for the frontend application. One of the backend services is Azure Functions API providing telemetry (getTelemetry) and telemetry history (getTelemetryHistory) data from Blob Storage. The second one is FastAPI inference service hosted via Azure Container Apps with three endpoints: predict endpoint providing results of the inference and performance metrics, health endpoint used to check whether the service works and validate endpoint providing information about enough data presence.

Data retrieval happens using scheduling with periodic requests to the endpoints for live telemetry data and predictions.

### **4.6.2 Data Mapping & Transformation Layer**

A data transformation process transforms unstructured API responses into structured dashboard data. The health score of the machine is obtained through normalization and expressed in percentage terms based on the extent of the reconstruction error in relation to the anomaly threshold.

The severity of the fault will be categorized as low, moderate, or high based on the extent of the difference between the reconstruction error and the anomaly threshold. System status will be categorized as normal, warning, or critical using inference outputs and early signs of degradation. Degradation timeline will be developed from window error values and key events like the occurrence of anomalies and faults.

Telemetry data will be transformed using vibration calculation based on acceleration across three axes, as well as temperature resolution. Maintenance advice will be generated in relation to the system state.

# Chapter 5

## Deployment and Validation

### 5.1 Deployment Overview

The system deployment spans three independently hosted components:

- **Inference Backend** — Dockerized FastAPI application hosted on Azure Container Apps
- **Serverless API Layer** — Azure Functions application serving telemetry data
- **Frontend** — React application served locally via Vite during validation

Each part of the system interacts with one another using HTTPS and can be configured independently using environment variables. The architecture is cloud native whereby compute, storage, and networking are all provisioned via Microsoft Azure.

### 5.2 Containerization of the Inference API

#### 5.2.1 Docker Image Configuration

The inference backend is packaged as a Docker container using the following configuration:

- **Base Image:** `python:3.11-slim`
- **Exposed Port:** `8000`
- **Entry Point:** Uvicorn ASGI server running the FastAPI application
- **Build Process:** Dependencies installed from `requirements.txt` during image build

The `python:3.11-slim` base image was selected for its minimal footprint while retaining full compatibility with the scientific Python ecosystem.

## 5.2.2 Application Dependencies

The following key libraries are included in `requirements.txt`:

- **FastAPI + Uvicorn** — HTTP application server and ASGI runtime
- **TensorFlow / Keras** — LSTM Autoencoder loading and inference
- **PyTorch** — CNN fault classifier loading and inference
- **scikit-learn + joblib** — StandardScaler deserialization
- **NumPy, pandas, SciPy** — Signal processing and feature extraction
- **Azure Blob Storage SDK** — Runtime download of model artifacts and telemetry data

## 5.2.3 Model Artifact Loading at Startup

On container startup, a FastAPI startup event handler executes the following steps:

- Downloads LSTM Autoencoder weights (`.h5`) from Azure Blob Storage — loaded via Keras
- Downloads StandardScaler (`.save`) — deserialized via joblib

- Downloads CNN classifier weights (.pth) — loaded into FaultCNN PyTorch architecture, set to eval mode
- Downloads anomaly threshold (.npy) — loaded as float; falls back to default value of **0.40** if unavailable
- All artifacts cached as module-level globals — subsequent requests reuse loaded models without re-downloading
- TensorFlow thread parallelism set to 1 to prevent CPU contention in containerized environment

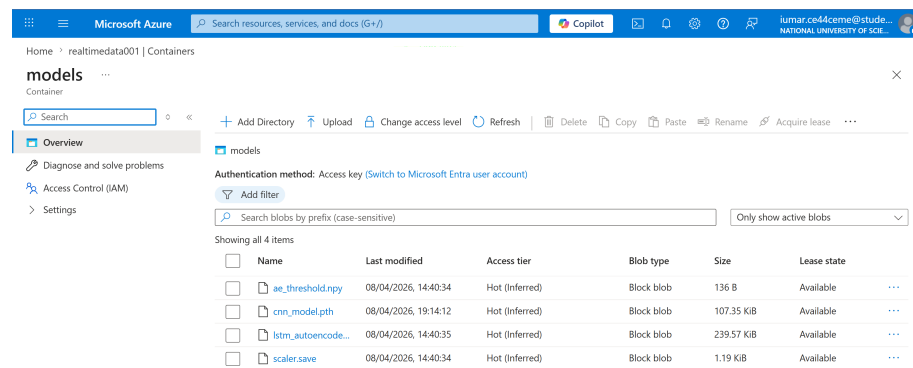


Figure 5.1: Model Artifacts Stored in Azure Blob Storage

## 5.2.4 FastAPI Endpoints

The application exposes the following REST endpoints:

| Endpoint  | Method | Description                                                  |
|-----------|--------|--------------------------------------------------------------|
| /         | GET    | Root health check                                            |
| /health   | GET    | Returns {"status": "healthy"}                                |
| /validate | GET    | Checks data sufficiency in Blob Storage                      |
| /predict  | GET    | Executes full inference pipeline, returns prediction payload |

Table 5.1: FastAPI Inference API Endpoints

## 5.3 Azure Cloud Deployment

### 5.3.1 Azure Container Registry

The Docker image is registered in **Azure Container Registry (ACR)** before deployment.

The steps followed are:

- Docker image built locally using Docker CLI
- Image tagged with the ACR login server address
- Image pushed to the private ACR repository
- ACR serves as the authoritative image source for Container Apps deployment

### 5.3.2 Azure Container Apps

The inference container is deployed on **Azure Container Apps** with the following configuration:

- **Public HTTPS Endpoint:** `https://test001.delightfulgrass-e114628e.southe`

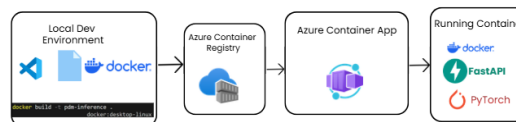


Figure 5.2: Real-Time ML Inference Pipeline diagram

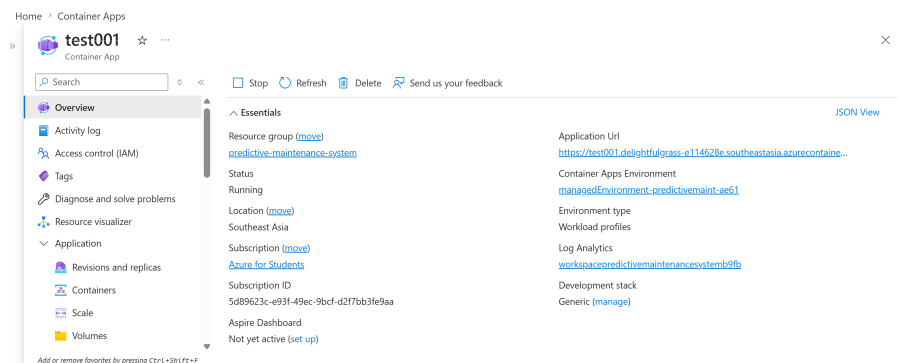


Figure 5.3: Inference API Deployed on Azure Container Apps

## 5.4 Azure Functions Deployment

Two separate Azure Function Apps are deployed:

### Function App 1 — IoT Ingestion Function

- **Trigger Type:** Event Hub trigger
- **Binding:** Connected to IoT Hub's Event Hub-compatible endpoint
- **Responsibility:** Parses incoming JSON telemetry, validates fields, appends records to telemetry CSV in Blob Storage
- **Activation:** Fires automatically on each arriving IoT Hub message

### Function App 2 — Telemetry API Function

- **Trigger Type:** HTTP trigger
- **Public Endpoint:** `https://pm-api-demo-d8gmgvfvinc2e5ft.southeastasia-01.azurefunctions.azurewebsites.net`
- **Endpoints Exposed:**
  - `GET /api/getTelemetry` — returns latest telemetry record
  - `GET /api/getTelemetryHistory?limit={n}` — returns `n` historical records
- **Data Source:** Reads directly from telemetry CSV in Azure Blob Storage

## 5.5 Frontend Deployment

The React frontend is built using **Vite** and served locally during the validation phase.

### Configuration:

- Inference API URL configured via `.env` file:

- Azure Functions telemetry endpoints referenced directly in API service layer
- Vite development server runs on `localhost:5173` or `localhost:5174`

## 5.6 Dashboard Walkthrough

The live monitoring dashboard presents a unified, continuously updated view of machine health status, sensor telemetry, and inference results organized across several panels.

### 5.6.1 Dashboard Header

The topmost section of the dashboard displays:

- Application title: **Predictive Maintenance**
- Live status indicator with last updated timestamp
- Data source selector confirming **Live Stream** mode

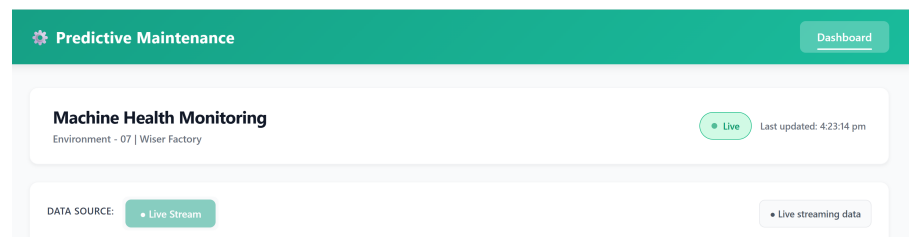


Figure 5.4: Dashboard Header with Live Stream Indicator

### 5.6.2 System Status Panel

The status panel presents the primary inference output:

- Fault Detected or not
- Backend Status
- Active Fault Type
- Health Score

- Reconstruction Error / Threshold
- First Anomaly Detected (T1)
- Confirmed Fault (T2)
- Severity
- F1 Normal / F1 Fault / F1 Macro

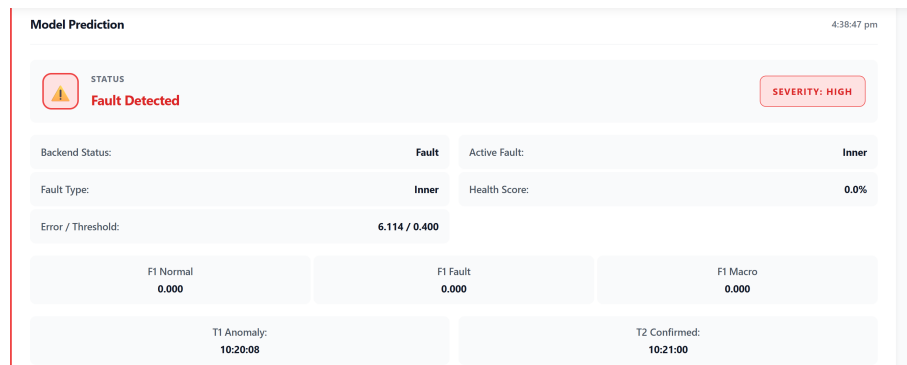


Figure 5.5: System Status Panel Showing Inner Race Fault Detection

### 5.6.3 Machine Health Metrics Panel

Displays instantaneous sensor readings retrieved from the Azure Functions telemetry API:

- Vibration RMS
- Temperature
- Current



Figure 5.6: Machine Health Metrics Panel

### 5.6.4 Rolling Statistics Panel

Presents statistical summary of sensor signals over the current observation window:

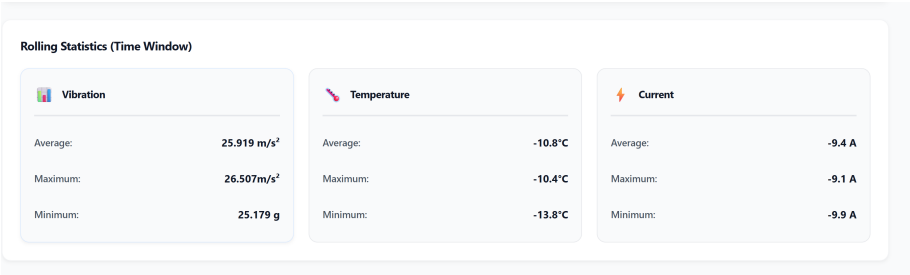


Figure 5.7: Rolling Statistics Panel Caption

### 5.6.5 Live Sensor Telemetry Charts

Time-series charts rendered for the observation window:

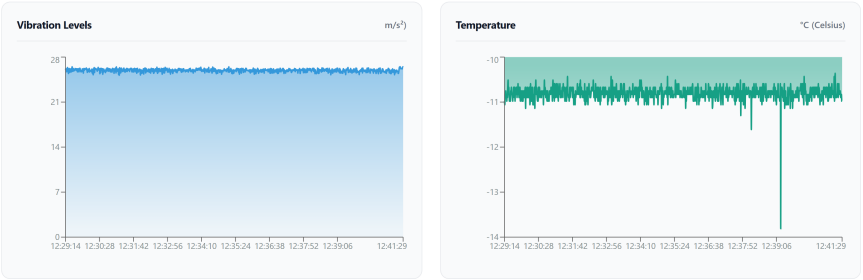


Figure 5.8: Machine Health Metrics Panel

### 5.6.6 Reconstruction Error Chart

The most detailed inference visualization on the dashboard:

- **X-axis:** Timestamp (window-aligned)
- **Y-axis:** Mean Absolute Error (MAE)
- **Blue line:** Per-window LSTM Autoencoder reconstruction error
- **Red dashed line:** Anomaly detection threshold (0.400)
- **T1 marker** — first anomaly window detected

- **T2 marker** — tenth consecutive anomaly, fault confirmed
- **Reconstruction error**

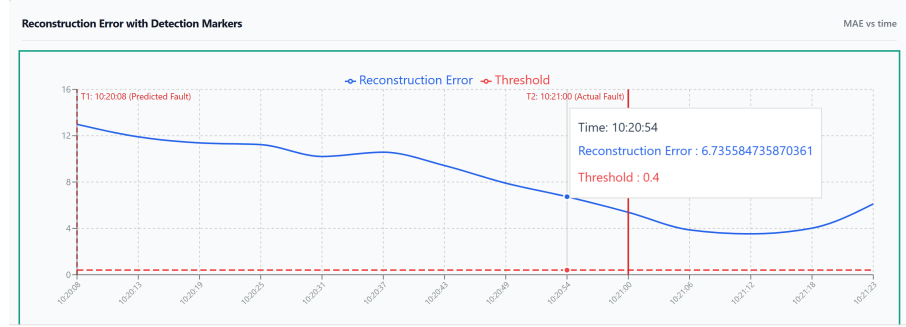


Figure 5.9: Reconstruction Error Chart with Fault Detection Markers

## 5.7 Model Performance Evaluation

This section presents the performance of the proposed predictive maintenance system across three evaluation scenarios: (i) developed dataset evaluation with DTW mapping, (ii) NASA IMS benchmark validation, and (iii) real-time hardware validation.

### 5.7.1 Evaluation on Developed Dataset

The model was first evaluated on the custom concatenated dataset. In each case, the reconstruction error remains consistently below the threshold during normal operation samples and rises sharply above it as soon as faulty samples are introduced, triggering fault confirmation via the K-consecutive mechanism.

Figure 5.10 illustrates this behavior for an inner race fault, where a distinct spike in reconstruction error marks the onset of the fault. DTW alignment maps this anomaly to approximately 50.64 hours prior to failure on the NASA IMS reference timeline, and the CNN correctly classifies the fault as an inner race defect.

Similarly, Figure 5.11 presents the reconstruction error pattern for an outer race fault. A sharp deviation indicates fault occurrence, which is aligned using DTW to approximately

50.33 hours before failure, with the CNN accurately identifying it as an outer race fault.

Figure 5.12 further demonstrates the model's response to a rolling element fault, where a pronounced increase in reconstruction error signifies the fault onset. DTW alignment places this detection at approximately 100.72 hours prior to failure on the NASA IMS timeline, and the CNN correctly classifies it as a rolling element defect.

### 5.7.1.1 Inner Race Fault

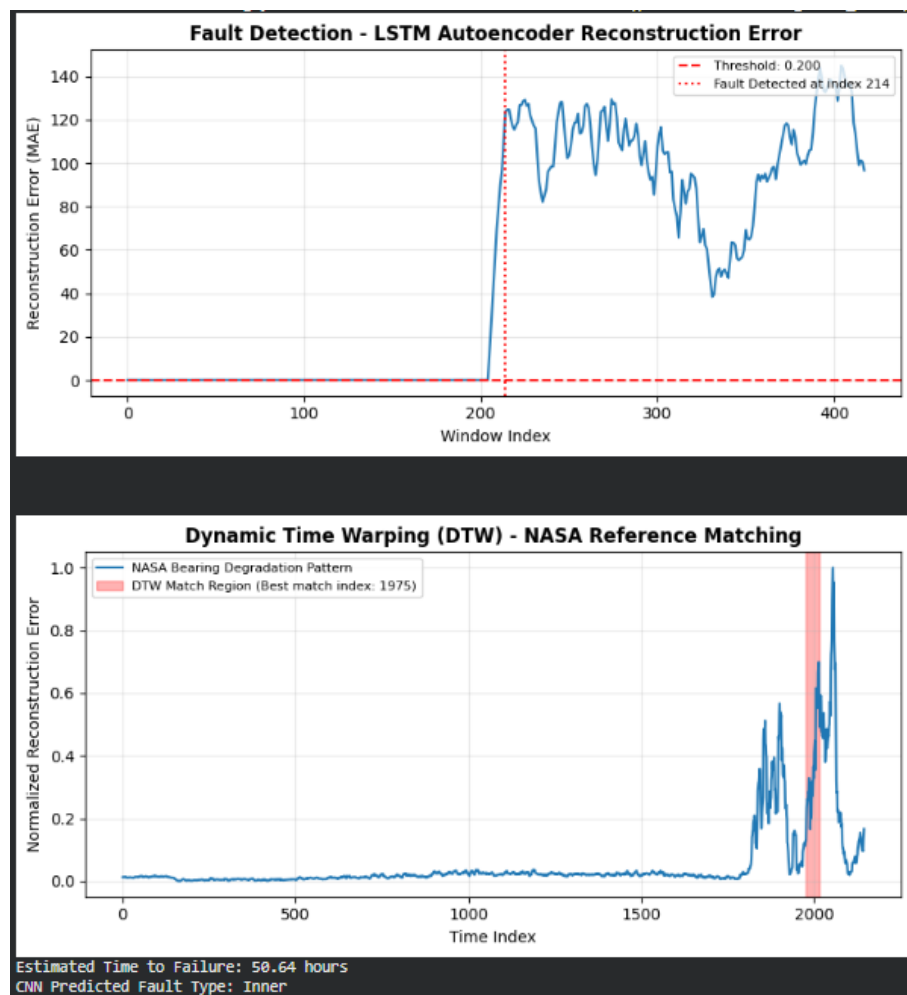


Figure 5.10: Reconstruction error highlighting inner race fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 50.64 hours prior to failure and CNN confirming the fault type.

### 5.7.1.2 Outer Race Fault

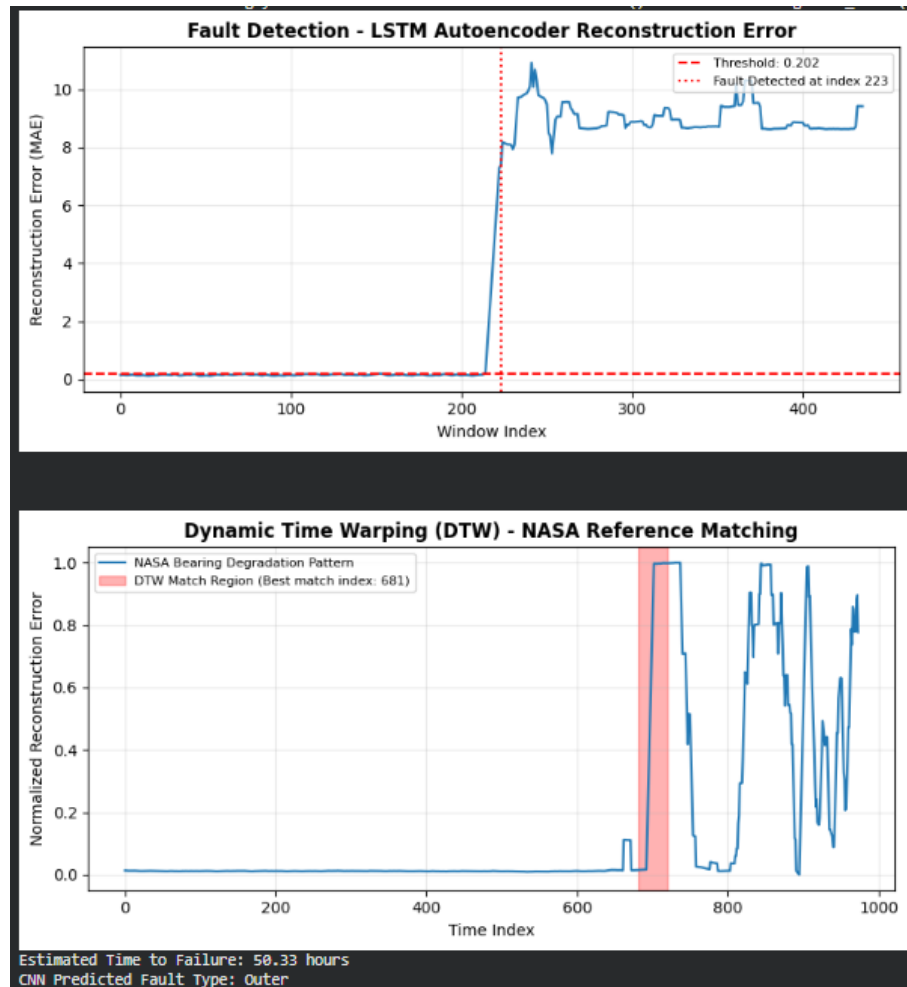


Figure 5.11: Reconstruction error highlighting outer race fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 50.33 hours prior to failure and CNN confirming the fault type.

### 5.7.1.3 Rolling Element Fault

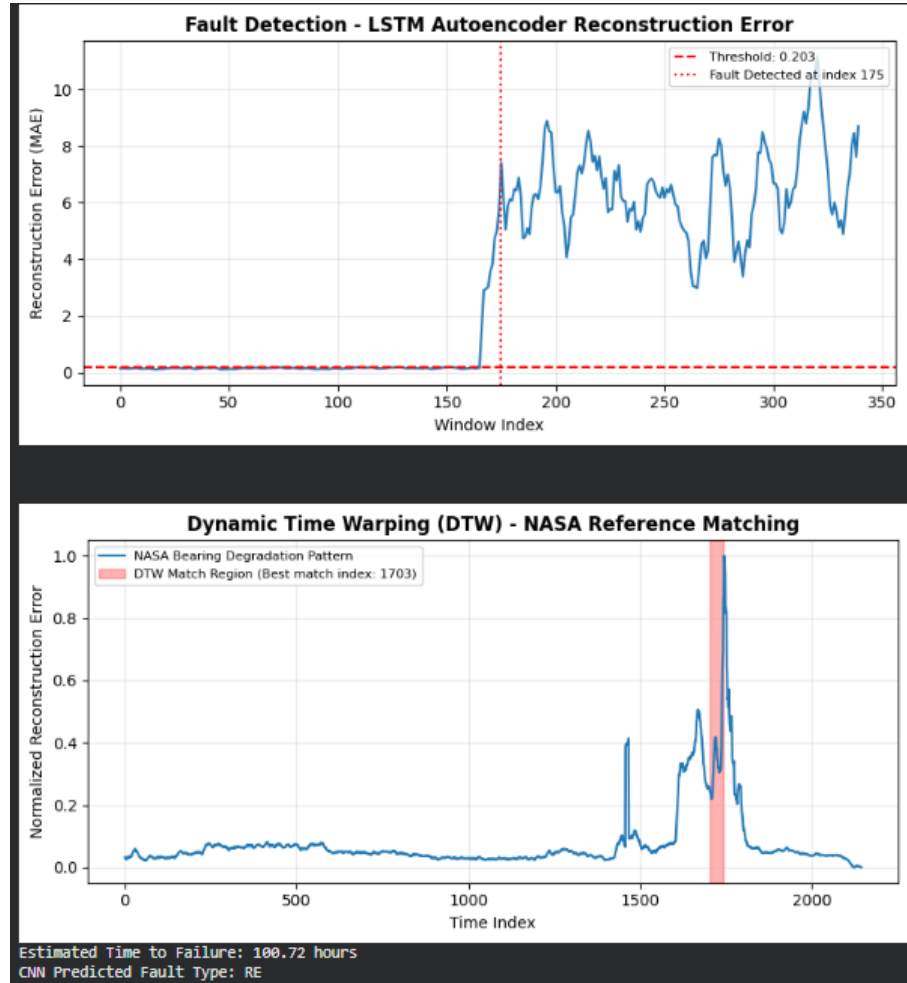


Figure 5.12: Reconstruction error highlighting rolling element fault detection, with DTW-based temporal alignment indicating fault occurrence approximately 100.72 hours prior to failure and CNN confirming the fault type.

Since the developed dataset is an artificially concatenated sequence rather than a true run-to-failure experiment, it carries no inherent timing information about when failure would actually occur. To estimate how early the detected fault maps relative to a real failure point, Dynamic Time Warping (DTW) was used to align the detected fault index from each concatenated dataset to the corresponding NASA IMS run-to-failure signal. The DTW alignment results, shown alongside each reconstruction error plot above, indicate that the detected fault point maps to a region close to the actual failure point in the NASA timeline. This is not a limitation of the model but of the dataset: the faulty bearings

used on the test rig contain direct surface defects rather than micro-faults that gradually develop over time, meaning the fault signal appears abruptly rather than progressively. This distinction has been discussed in previous sections.

## 5.7.2 Evaluation on NASA IMS Dataset

To validate that the problem is our own developed dataset and to ensure that the LSTM Autoencoder functions correctly on real degradation data with known failure timings, the same model architecture was independently evaluated on the NASA IMS run-to-failure dataset. Here, the reconstruction error remains low during the extended healthy operation period at the start of each run. As bearing degradation progresses, the error begins to approach and eventually consistently exceed the threshold, at which point the K-consecutive mechanism confirms the fault.

As shown in Figure 5.13, the model detects an inner race defect in Set 1 Bearing 3, with fault confirmation occurring approximately 16.01 days before failure. Figure 5.14 illustrates a rolling element defect in Set 1 Bearing 4, where the fault is identified approximately 18.24 days prior to failure. Similarly, Figure 5.15 presents an outer race defect in Set 2 Bearing 1, with fault confirmation approximately 3.05 days before failure.

### 5.7.2.1 Inner Race Fault – Set 1 Bearing 3

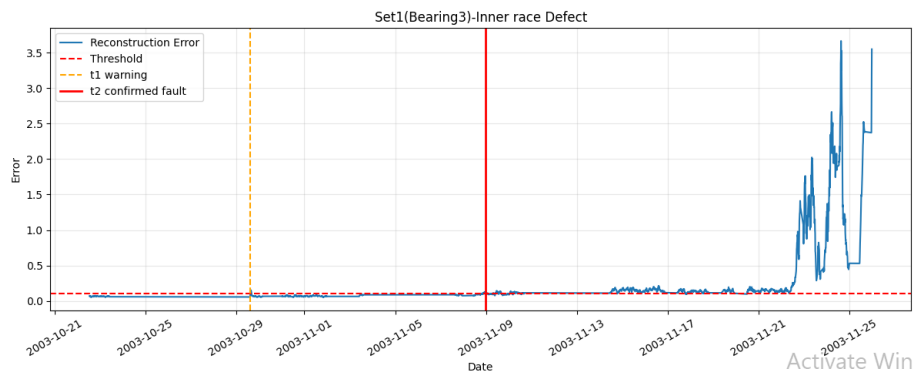


Figure 5.13: Reconstruction error for Set 1 Bearing 3 (inner race defect). Fault confirmed approximately 16.01 days before failure.

### 5.7.2.2 Rolling Element Fault – Set 1 Bearing 4

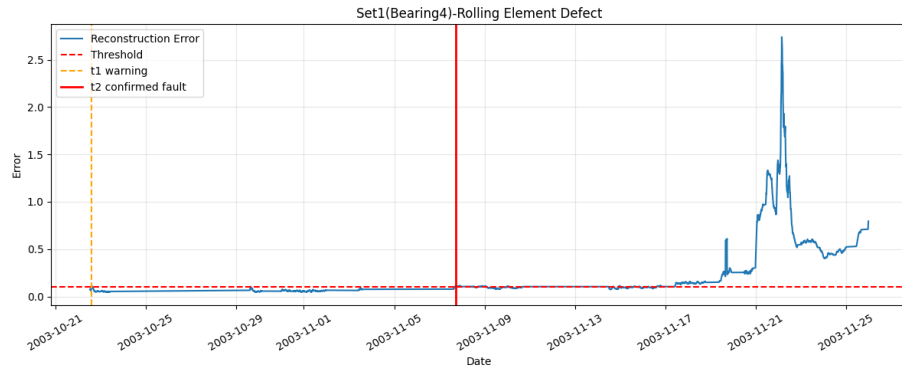


Figure 5.14: Reconstruction error for Set 1 Bearing 4 (rolling element defect). Fault confirmed approximately 18.24 days before failure.

### 5.7.2.3 Outer Race Fault – Set 2 Bearing 1

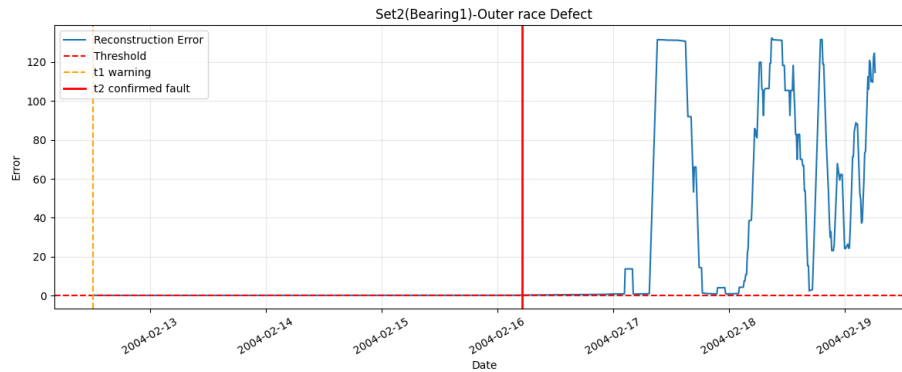


Figure 5.15: Reconstruction error for Set 2 Bearing 1 (outer race defect). Fault confirmed approximately 3.05 days before failure.

Across all three NASA IMS bearings, the model successfully confirmed faults well ahead of the actual failure point, providing advance warning ranging from approximately 3 (in the case of 7-day signals) to 18 days.

Additionally, a NASA bearing that never developed a defect was also evaluated. As illustrated in Figure 5.16, the reconstruction error for this bearing shows a gradual increase over time but consistently remains below the fault threshold. It also does not satisfy the K-consecutive criterion, resulting in no fault confirmation and correctly reflecting a healthy operating condition.

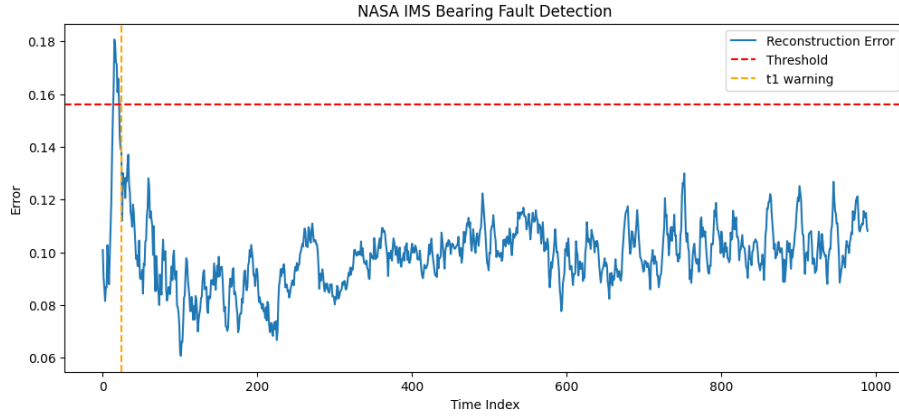


Figure 5.16: Reconstruction error for a healthy bearing, showing a gradual increase over time that remains below the fault threshold and does not trigger fault confirmation.

### 5.7.3 Real-Time Hardware Validation

The system was finally validated using live vibration data streamed from the ESP32 edge node mounted on the test rig. Due to physical constraints, only one bearing condition could be attached to the rig at a time; therefore, data from normal and faulty conditions cannot be concatenated at runtime. Accordingly, the reconstruction error is either entirely below the threshold for a healthy bearing or entirely above it for a faulty bearing from the very first window.

As shown in Figure 5.17, under normal operating conditions the reconstruction error remains consistently below the threshold throughout the streaming process, and no fault is confirmed.

In contrast, Figure 5.18 illustrates the faulty operating condition, where the reconstruction error exceeds the threshold from the first window. A fault is then confirmed immediately after satisfying the K-consecutive criterion of 10 anomalous sequences, demonstrating reliable real-time detection performance.

### 5.7.3.1 Normal Operating Condition

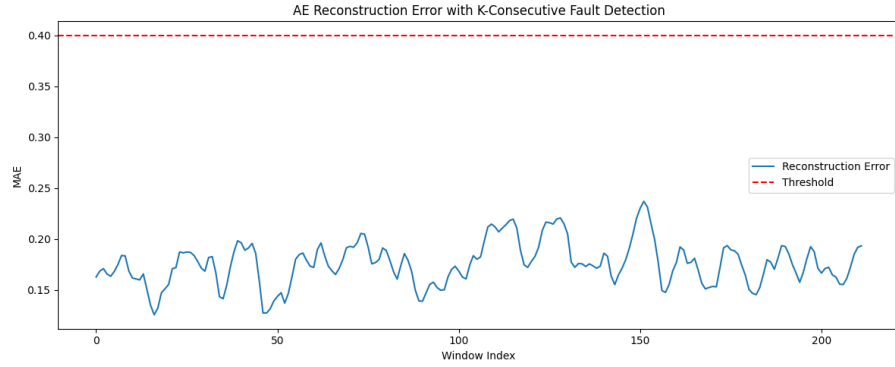


Figure 5.17: Real-time reconstruction error under normal operating condition. The error remains consistently below the threshold throughout and no fault is confirmed.

### 5.7.3.2 Faulty Operating Condition

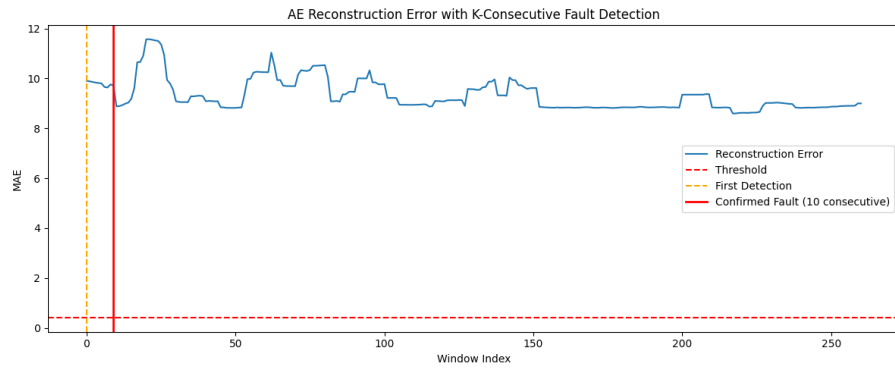


Figure 5.18: Real-time reconstruction error under faulty operating condition. The error exceeds the threshold from the first window and a fault is confirmed immediately after 10 consecutive anomalous sequences.

The real-time results confirm that the fully deployed end-to-end pipeline behaves correctly under live sensor data. The model cleanly separates normal from faulty conditions, and the CNN classifier correctly identifies the fault type upon confirmation.

## 5.7.4 Discussion

The system demonstrates strong performance across both detection and classification tasks. For fault detection, the LSTM Autoencoder flags bearing degradation at approx-

imately 47% of the bearing's operational lifetime, providing the maintenance engineer with a confirmed alert while more than half of the bearing's remaining service life is still available. For fault classification, the 1-D CNN achieves an accuracy of 86.8% and an F1-score of 92.9% across the three fault classes, indicating reliable and balanced classification performance without bias toward any single fault type.

## 5.8 System Validation

### 5.8.1 Inference API Validation

The deployed inference API was validated by testing each endpoint:

#### **/health endpoint:**

- Returns: {"status": "healthy or faulty"}
- Confirms container is running and reachable over HTTPS

#### **/validate endpoint:**

- Returns row count, window count, sequence count
- Confirms data sufficiency for inference

#### **/predict endpoint:**

- Returns full prediction payload including:
  - final\_status: Fault / Normal
  - fault\_type: Inner / Outer / RE
  - health\_score: normalized 0–100%
  - errors: per-window reconstruction error array
  - degradation\_curve: smoothed error series

- $t_1, t_2$ : anomaly onset timestamps
- $f1\_normal, f1\_fault, f1\_macro$ : model performance metrics

### 5.8.2 End-to-End Pipeline Validation

The complete pipeline was validated by running the system under a controlled fault condition.

#### Test Scenario:

- ESP32 node transmitting live vibration data from a bearing with an **Inner race fault**
- Data continuously ingested via IoT Hub → Azure Function → Blob Storage
- Inference API polling Blob Storage and running prediction on accumulated data

#### Observed Results:

| Metric               | Value    |
|----------------------|----------|
| Final Status         | Fault    |
| Fault Type           | Inner    |
| Health Score         | 0.0%     |
| Reconstruction Error | 6.114    |
| Anomaly Threshold    | 0.400    |
| First Anomaly (T1)   | 10:20:08 |
| Confirmed Fault (T2) | 10:21:00 |
| Severity             | HIGH     |

Table 5.2: End-to-End Validation Results

#### Validation Outcome:

- System correctly identified the presence of a fault
- CNN classifier correctly identified the fault as an **Inner race fault**
- Reconstruction error (6.114) was **15.3**× above the anomaly threshold (0.400), confirming a severe fault condition

- T1 to T2 detection window spanned **52 seconds**, demonstrating rapid fault confirmation
- Dashboard updated in real time reflecting the fault state across all panels

# Chapter 6

## Conclusion and Future Work

### 6.1 Conclusion

The objective of this project was the design, development, and validation of a fully-fledged IoT-based predictive maintenance system for rotary machines. The ESP32 sensor sends the triaxial vibration, temperature, and current data through an IoT cloud service. The containerized two-stage machine learning pipeline in the Azure cloud, comprised of the LSTM Autoencoder for anomaly detection and the 1-D CNN for fault detection, analyzes the data, and publishes the results on the React web application interface. The system achieved high reliability during fault detection and classification using the NASA IMS benchmark, custom degradation dataset, and real-life test rig setup.

### 6.2 Future Work

Perhaps the most significant hardware improvement to make is running the inference on the edge device using TensorFlow Lite to remove the reliance on Wi-Fi. In addition, adding surface temperature and motor current as additional inputs for the LSTM Autoencoder will provide an even more accurate picture of the machine's condition, as both features provide diagnostic information that cannot be gathered by just measuring vibrations

especially when the machines operate at higher loads.

From the perspective of software and implementation, adopting the new polling mechanism to event-driven triggers by using Azure Event Grid and cache intermediate processing results between invocations will decrease latency times to well under a second, making the system suitable for use in critical situations where the timely notification of faults is essential. Finally, performing fault experiments on internal motor bearings that have yet to be covered by this project will greatly enhance the potential impact of the research, as the drive-end bearing is one of the most common parts that get replaced in rotating machinery.

# References

- [1] R. B. Randall and J. Antoni, “Rolling element bearing diagnostics—a tutorial,” *Mechanical Systems and Signal Processing*, vol. 25, no. 2, pp. 485–520, 2011.
- [2] J. Antoni, “The spectral kurtosis: a useful tool for characterising non-stationary signals,” *Mechanical Systems and Signal Processing*, vol. 20, no. 2, pp. 282–307, 2006.
- [3] F. Immovilli, A. Bellini, R. Rubini, and C. Tassoni, “Detection of generalized-roughness bearing fault by spectral kurtosis estimation of vibration signal,” *IEEE Transactions on Industrial Electronics*, vol. 56, no. 11, pp. 4710–4717, 2009.
- [4] W. Caesarendra and T. Tjahjowidodo, “A review of feature extraction methods in vibration-based condition monitoring and its application for degradation trend estimation of low-speed slew bearing,” *Machines*, vol. 5, no. 4, p. 21, 2017.
- [5] A. Widodo and B.-S. Yang, “Support vector machine in machine condition monitoring and fault diagnosis,” *Mechanical Systems and Signal Processing*, vol. 21, no. 6, pp. 2560–2574, 2007.
- [6] M. Cerrada *et al.*, “A review on data-driven fault severity assessment in rolling bearings,” *Mechanical Systems and Signal Processing*, vol. 99, pp. 169–196, 2018.
- [7] Y. Lei, B. Yang, X. Jiang, F. Jia, N. Li, and A. K. Nandi, “Applications of machine learning to machine fault diagnosis: A review and roadmap,” *Mechanical Systems and Signal Processing*, vol. 138, p. 106587, 2020.

- [8] P. Konar and P. Chattopadhyay, "Bearing fault detection of induction motor using wavelet and support vector machines," *Applied Soft Computing*, vol. 11, no. 6, pp. 4203–4211, 2011.
- [9] M. N. Uddin and M. R. Islam, "Bearing fault diagnosis using a random forest classifier," *IETE Journal of Research*, vol. 68, no. 5, pp. 3333–3341, 2020.
- [10] L. Wen *et al.*, "A new convolutional neural network-based data-driven fault diagnosis method," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 7, pp. 5990–5998, 2017.
- [11] W. Zhang *et al.*, "A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals," *Sensors*, vol. 17, no. 2, p. 425, 2017.
- [12] X. Guo, L. Chen, and C. Shen, "Hierarchical adaptive deep convolution neural network and its application to bearing fault diagnosis," *Measurement*, vol. 93, pp. 490–502, 2016.
- [13] Z. Chen and W. Li, "A deep learning method for bearing fault diagnosis based on cyclic spectral coherence and convolutional neural networks," *Mechanical Systems and Signal Processing*, vol. 140, p. 106683, 2020.
- [14] C. Zhang, P. Lim, A. Qin, and K. C. Tan, "Long short-term memory for machine remaining life prediction," *Journal of Manufacturing Systems*, vol. 48, pp. 78–90, 2018.
- [15] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, "Deep learning and its applications to machine health monitoring," *Mechanical Systems and Signal Processing*, vol. 115, pp. 213–237, 2019.
- [16] P. Malhotra *et al.*, "Lstm-based encoder-decoder for multi-sensor anomaly detection," in *ICML Anomaly Detection Workshop*, 2016.

- [17] R. Chalapathy and S. Chawla, “Deep learning for anomaly detection: A survey,” *arXiv preprint arXiv:1901.03407*, 2019.
- [18] E. Principi, D. Rossetti, S. Squartini, and F. Piazza, “Unsupervised electric motor fault detection by using deep autoencoders,” *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 2, pp. 441–451, 2019.
- [19] D. Park, H. Kim, and C. C. Kemp, “Multimodal anomaly detector in robot-assisted feeding using an lstm-based variational autoencoder,” in *IEEE Robotics and Automation Letters*, vol. 4, pp. 1590–1597, 2018.
- [20] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed, “Deepant: A deep learning approach for unsupervised anomaly detection in time series,” *IEEE Access*, vol. 7, pp. 1991–2005, 2019.
- [21] J. Lee, H. Qiu, G. Yu, and J. Lin, “Bearing data set.” IMS, University of Cincinnati, NASA Ames Prognostics Data Repository, 2007.
- [22] Case Western Reserve University, “Bearing data center.” <https://engineering.case.edu/bearingdatacenter>, 2000.